

MANUAL

Every knob in AlphaNode, explained in plain language.

What each panel shows, what every setting actually changes, and what each leaderboard column measures — with the shipped default, the range the app allows, and the traps worth knowing before you turn something up. Read it end to end once, or jump to the knob you are looking at right now.

CONTENTS

- 01 The basics — what you are mining, and what the main window shows
 - 02 Pairs, timeframe and market data
 - 03 The search settings — how hard and how long it looks
 - 04 The engine knobs — simulation, formula shape, breeding and fitness
 - 05 Date segments, and the honesty discipline
 - 06 The leaderboard column by column — and what you can do with a formula
-

Part 01

The basics — what you are mining, and what the main window shows

AlphaNode is a formula miner for crypto perpetual futures. It searches, over and over, for small maths expressions ("alphas") that turn raw candles into a position in each coin, scores them on history, and keeps the ones that survive — you press Start and walk away. This block explains what the thing being mined actually is, what happens when you press Start, how the app splits history into an honest training / checking / untouched-exam layout, and what every card and button on the main window is telling you. Nothing here is a setting you have to tune; the knobs come later.

A formula (an "alpha")

A short maths expression over the price tables — a real one from a library looks like `ema:20(ts_delta:14(cs_zscore(cs_demean(add(ts_max:30(volume),ret))))`). The number after a colon is a lookback in bars. The expression is evaluated once per coin per bar and produces a single number: positive means "be long this coin", negative means "be short", bigger magnitude means "a bigger share of the book". The leaves are the only 12 things it can look at (close, open, high, low, volume, ret, vwap, range, body, dvol, logret, funding) and the branches are 25 operators: arithmetic (add/sub/mul/div/pmin/pmax), shape-benders (neg/sign/abs/slog/tanh), lookback operators over 2–200 bars of one coin's own history (ts_mean, ts_std, ts_zscore, ts_min, ts_max, ts_delta, ts_delay, ts_sum, ts_roc, ema) and cross-sectional operators that compare all your coins against each other on the same bar (cs_rank, cs_zscore, cs_demean, cs_scale). There are no plain numbers in the grammar — every leaf is a data series, and the only digits you will ever see are lookback windows.

HOW TO SET IT

Read a formula as a recipe, not as a price target. Everything downstream of the number is fixed and identical for every alpha: each coin's number is divided by that coin's recent volatility (a 30-bar rolling standard deviation, floored so a dead-calm coin can't explode the weight), then the whole row is divided by the sum of the absolute values, giving target weights that always add up to 100% of the book; that book is then scaled to your target volatility, held through a 10% no-trade band, and charged your fee on turnover — with perpetual funding paid or received on whatever is held. So the only things the search can change are the SIGN and the RELATIVE SIZE of the numbers across coins and across time. Prefer the shorter formula when two score alike — you can actually reason about it.

Watch out. Absolute scale is a no-op, and the search still spends nodes on it. `add(x,x)` is exactly twice `x`, which after the normalisation step is the identical strategy — but it costs two extra nodes of complexity penalty. Real champions in the shipped library contain exactly this (`add(volume,volume)` inside one of them). Cross-sectional operators also need a real cross-section: with 1-2 pairs in your universe, `cs_rank` and `cs_demean` collapse to near-constants and those families die out.

A round

default: population 200 · 25 generations · Hall of Fame 15 · pause 5 s · explore every 4th round · warm-start from library ON (note: `evolution/config.ini` says 30 generations, but the app's own default and what a fresh install runs is 25)

range: population 4–4,000 · generations 1–500 · Hall of Fame 1–100 · pause 0–3,600 s · explore every 1–100 rounds (all are spinboxes: the arrows respect the range, a typed value is not clamped)

One round is one complete evolutionary search, run start to finish inside the node process. It builds a population of candidate formulas (200 by default), scores every one of them, breeds the survivors for 25 generations, then fine-tunes the lookback windows of its top 5 finishers (trying each window at $\times 0.8$ and $\times 1.25$, up to three passes, keeping only changes that score better), and finally hands back a Hall of Fame — a shortlist of up to 15 champions kept deliberately un-alike. Any champion whose formula is not already in your library gets appended to the library file. Then the node pauses (5 seconds by default) and starts the next round with a new seed.

HOW TO SET IT

Rounds are the unit of progress. A round costs at most $\text{population} \times \text{generations} = 5,000$ formula evaluations, and usually noticeably fewer, because formulas that repeat within the round are only scored once; the live log prints the real count per round. Wall-clock time runs from well under a minute to several minutes depending on your CPU share, universe size and timeframe. By default every 4th round starts from a fresh random population (hunting new families); the other three warm-start from the champions you already have and try to improve on them — nothing is deleted either way, exploring just ignores the library for that round. Let it run for tens of rounds before judging anything. The live log announces each round's mode ("explore" or "refine") and its result.

Watch out. Warm-start only ever sees formulas found in the CURRENT session. Until you activate the node, everything on disk is sealed, so the pool of champions to refine starts empty and the node quietly falls back to exploring. Worse, the pool is the node's own top list, capped by the "Leaderboard size" setting (20): if 20 sealed champions from earlier sessions already hold those slots with higher scores, this session's new finds never get in — and refine never runs at all, however long you leave it. Activating the node fixes this permanently.

TRAIN / VAL / TEST

default TRAIN 2019-09-05 · VAL 2021-11-01 · TEST 2023-01-01 · TEST end 2026-07-05

History is cut into three consecutive date ranges. TRAIN (default 2019-09-05 → 2021-11-01) is where evolution does its work. VAL (2021-11-01 → 2023-01-01) is a second stretch that breeding never optimises on directly, used to check that a formula was not just memorising TRAIN. TEST (2023-01-01 → 2026-07-05) is held out entirely: no part of the search reads it — not the fitness, not the selection, not the warm-start seeding. Its numbers are computed and carried along purely so you can look at them afterwards.

HOW TO SET IT

Treat the TEST columns as a report card you are allowed to read once you have decided what to keep. The moment you start picking alphas BECAUSE their TEST number is high, TEST stops being out-of-sample and becomes just another training set — that is why the app marks every TEST-flavoured action with a $\triangle$ (clicking the TEST OOS column header, choosing "by TEST" in the Portfolio card). Picking a different timeframe rewrites all four dates to that bar size's recommended window, because intraday history is shorter and denser. If you want a genuinely honest verdict after you have already peeked, use the FORWARD TRACK card: it steps forward on bars that did not exist when the formula was mined.

Watch out. "TEST end" is a hard-coded calendar date (2026-07-05 for daily, 2026-07-01 for the three intraday timeframes) baked into the defaults, config.ini and the timeframe table alike. Start refreshes your market data whenever the newest bar in the snapshot is more than 5 days old — but everything after TEST end falls outside all three segments and is never simulated. If you have been running for months and never edited that field, the fresh data you keep downloading is doing nothing.

Fitness — the number the search maximises

default: min(TRAIN, VAL) Sharpe · complexity penalty 0.010 per node · correlation threshold 0.70 · similarity penalty 0.5 · concentration penalty 0.3 below 3 effective positions · minimum activity 10% of TRAIN and of VAL bars

range: complexity penalty 0.0–1.0 · correlation threshold 0.0–1.0 · similarity penalty 0.0–2.0 (the concentration penalty and the 10% activity floor have no GUI field at all)

By default fitness is the WORSE of two Sharpe ratios: $\min(\text{TRAIN Sharpe}, \text{VAL Sharpe})$. A Sharpe ratio here is average return divided by the wobble of those returns, both scaled to a year — roughly "how much reward per unit of shakiness". Taking the worse of the two eras is deliberate: a formula is only as good as its weakest era, so curve-fitting one bull market buys you nothing. On top of that the search subtracts three things while it breeds: 0.010 per node of formula size (simple beats complex), a diversity penalty of $0.5 \times$ how far the formula's return stream correlates above 0.70 with a champion already found, and a concentration penalty of $0.3 \times$ the shortfall when the book averages fewer than 3 meaningfully-sized positions across TRAIN+VAL (no one-coin bets). A formula is discarded outright unless it is actually in the market on at least 10% of TRAIN bars AND 10% of VAL bars — those never even get scored.

HOW TO SET IT

Fitness is the only score you can trust as un-peeked, because it is computed without TEST. But it is still a TRAIN/VAL number, and a high one is not a promise: in the library shipped with this repo the top rows score above +2 and go on to a NEGATIVE held-out TEST Sharpe. Treat fitness as "this survived two different eras" and let the TEST column and the forward track decide the rest. Note also that flat bars count: a strategy that sits out two-thirds of the period is measured over the whole calendar anyway, so "rarely trades but wins when it does" gets no free Sharpe bonus. TEST has no minimum-activity rule of its own, so a champion that barely traded during TEST is still kept and simply shows "—" in the TEST OOS column — an empty TEST cell is not a bad score, it is no score.

Watch out. The fitness you SEE is not the fitness the search ranks by. The number stored on every champion — the leaderboard's "fitness" column and the BEST FITNESS tile — is $\min(\text{TRAIN}, \text{VAL})$ with only the concentration penalty applied; the size and diversity penalties steer breeding inside a round and are then thrown away. That is why the live log's green "★ new best fit +1.62" line reads lower than the "✓ round N ... best fitness +1.81" summary for the very same formula: the ★ line carries the penalties, the summary does not.

► Start node (header, top right)

Starts the background search with whatever is currently in the settings panel. It first saves your settings, then checks your market-data snapshot: if the file is missing or unreadable, is missing one of your pairs, or has a newest bar more than 5 days old, it opens a console window and downloads exactly your pairs at your timeframe from Binance, then resumes Start by itself. Then it launches the node as a separate process, hands it every setting as an environment variable, and starts polling the node's status file every 1.5 seconds.

HOW TO SET IT

You do not need to prepare anything — defaults work and the data fetches itself on the first run. Press it once and leave it; the button greys out while the node lives. If a pair name is not a real Binance USDT-perpetual ticker, Start will download, notice the pair still missing, drop it from your universe with a warning, and carry on with the rest rather than looping. The node also publishes a live status page at <http://localhost:8787> — and it binds on all network interfaces, so anyone on your LAN who knows the port can read it.

Watch out. The mining node reads your settings **ONLY** at launch — they travel to it as environment variables in that one moment. Editing population, dates, fees or the universe while the node runs changes nothing about the search until you Stop and Start again. (The leaderboard's own analysis columns and the Portfolio build are a separate matter: those workers use whatever was last SAVED, and Start is one of the things that saves.)

■ Stop (header, top right)

Asks the node to shut down by sending it an interrupt. The node notices at the next generation boundary, unwinds, writes its final status and exits. The button greys itself out immediately, but the process can take a few seconds to a couple of minutes to actually die — the generation being evaluated has to finish first.

HOW TO SET IT

Stop before doing anything that rewrites your library: activating the node, loading a session and clearing history all refuse to run while a miner is appending to the same files. Everything already written to the library is safe; stopping never loses past rounds.

Watch out. The Stop tooltip says "the current round will finish" — it does not. The interrupt is raised inside the round, the round is abandoned, and its whole Hall of Fame is discarded before ever reaching the library file. If the live log shows a round most of the way through with a ★ new best, pressing Stop throws that champion away. Wait for the "✓ round N done" line in the log if you care about the round in flight.

● stopped / ● starting / ● running (the state pill)

The rounded badge at the top-left of the status card, showing what the node itself reports. Grey "● stopped" means no search is running; blue "● starting" means the node is booting — checking or downloading data and loading your existing library; green "● running" means it is mining rounds. To the right of the pill sits a one-line resource summary: CPU budget, how many of your cores that works out to, your universe and the effective target volatility.

HOW TO SET IT

"starting" can legitimately last a while on the first run of a timeframe (it may be downloading years of candles) or with a large library to load. If it sits there with nothing in the live log, open the status page on port 8787 or watch the data console window. When nothing is running, the resource line and the round ticker below it are prefixed with "last run —" so you do not mistake stale text for live telemetry.

Watch out. The pill mirrors the node's own status file, not the process. If the node was killed hard (a crash, a power cut, a task-manager kill) that file keeps its last written state — "running" — and the pill shows a green "● running" with no node behind it, and without the "last run —" prefix on the lines beside it. The giveaway is the Start button: if Start is clickable, nothing is mining.

⚙ Settings (header toggle)

```
default hidden (settings_open = false)
```

Shows and hides the left-hand settings panel, which holds every knob the engine understands — resources, pairs, timeframe, population, node mode, simulation costs, formula complexity, the breeding rules, the fitness penalties and the date segments. The panel is hidden on a fresh install, the button's label and outline turn accent-coloured while it is open, and the choice is remembered between launches. The pane is always exactly as wide as its own content; the split is deliberately not draggable.

HOW TO SET IT

Leave it closed for normal use — the dashboard is the working surface and the defaults are the ones the engine was tuned with. Open it when you want a different pair basket, a different timeframe, or a different date window. Every field carries a hover tooltip with a short explanation. At the bottom sit "Reset to defaults" and the red "Clear all history".

Watch out. "Reset to defaults" resets every search field but deliberately keeps two things: your theme and whether this panel is open. And "Clear all history" deletes the libraries and round history of EVERY timeframe at once, not just the one you are looking at, plus the built portfolio and its chart. Your settings, your starred favourites and your forward track survive it; your mined alphas do not, and nothing is backed up first — the dialog itself tells you to save a session if you want a way back.

📁 Sessions (header)

Your whole workspace as one file: the formula libraries of every timeframe, the round history, the forward track, the built portfolio and your settings, packed into a .tar.gz you can list, inspect, save and restore. Market data is not included (it re-downloads), and neither is your

subscription key — a session can safely travel to another machine or another person. Loading one replaces the current workspace; it refuses to run while the node, a forward step or a portfolio build is working.

HOW TO SET IT

Save a session before anything irreversible — clearing history, a big settings experiment, handing the machine to someone else. Double-click a row to see what is inside before you load it (how many alphas per timeframe, what the paper equity was, what portfolio was built). Restoring an older session does not rewind the forward track's clock: stepping resumes from the next closed bar and the gap stays visible in the history, exactly like a laptop that was simply switched off.

Watch out. The Sessions button's own tooltip says "an auto snapshot is taken every time the node stops". It is not — the app creates sessions only when you ask. The single automatic snapshot in the whole app is the "before-load" checkpoint taken when you LOAD a session, so the only safety net you get is the one for the operation you were already worried about.

Light / Dark (header switch)

default '' — follows the operating system on the very first launch

The appearance toggle. Flipping it re-paints the whole palette and rebuilds the window from scratch — that rebuild is what lets the parts that cannot restyle themselves live (the leaderboard table, the equity images, the chart canvas) come back in the right colours. Your leaderboard, portfolio chart and running services are restored immediately afterwards; nothing is lost and the node is untouched.

HOW TO SET IT

Harmless to flip at any time, including mid-search. Expect a visible flicker as the window is rebuilt; that is by design, not a crash. The equity PNGs are re-rendered on the new background rather than tinted.

Watch out. "Follow the OS" happens once, at startup: the resolved light/dark value is written straight into your in-memory settings and lands in the file the next time anything saves (pressing Start is enough). There is no "system" position on the switch to go back to, and "Reset to defaults" explicitly preserves the theme — so the only route back is closing the app and setting "theme" back to "" in gui_settings.json by hand.

Activate node / Unlock

Every formula the node mines is sealed before it touches your disk: the library stores encrypted text plus a public 12-character id, and the leaderboard shows the row's stored metrics but "locked — a subscription reveals the formula" instead of the maths. "Activate

node" (in the leaderboard card's header) takes your subscription key once, claims one of your plan's machine seats, unlocks EVERY sealed formula in every local library in one pass, and from then on the node mines in the open — re-checked against the licence server at each Start, so a lapsed subscription silently goes back to sealing. "Unlock" is the single-formula version, offered when you double-click a locked row.

HOW TO SET IT

Activate once per machine and forget it; the key is remembered so it is a single click thereafter. If the licence server is unreachable at Start, the node fails closed and seals that session's finds — they are not lost, just sealed, and the next successful activation reveals them too. Do it early, because sealed rows are inert to everything that has to re-simulate a formula: the leaderboard's computed columns, the Portfolio build, Serve and the forward track all need the plain text.

Watch out. Two real traps. First, activation rewrites the library files, so it refuses to run while the node is mining ("stop the node first") and Start refuses to launch a miner while an activation is in flight — stop, activate, then start. Second, the per-row "Unlock" is session-only: it reveals the formula in memory and the row shows its text, but the file on disk stays sealed, so the row is locked again after a restart. Only "Activate node" makes it permanent.

The status card — ROUNDS · FORMULAS TRIED · ALPHAS FOUND

The dashboard's masthead: the state pill, the resource line, four big tiles and a live log. All four tiles are read from the node's status file, not from your library. ROUNDS is how many complete searches have finished — the node resumes the count from its round history when it starts, so it keeps counting across restarts. FORMULAS TRIED is the lifetime total of distinct formulas evaluated, summed over rounds. ALPHAS FOUND is how many distinct formulas are in the library the node is currently mining into. Under the tiles, a single monospaced line tickers the current round and generation. The dashboard cards can be dragged into whatever order you like.

HOW TO SET IT

Watch ALPHAS FOUND versus ROUNDS to judge whether the search is still productive: early on, most rounds contribute several champions; once the library is mature, whole rounds pass with "none kept — the library held its bar" and that is normal, not a fault. FORMULAS TRIED grows by roughly population × generations per round minus repeats, plus the window fine-tuning candidates, and is the honest measure of how much compute you have spent.

Watch out. Switching timeframe in the panel does NOT move these three tiles. The leaderboard below re-reads that timeframe's own library file immediately, but the tiles keep showing the last run's numbers — of whichever timeframe ran last — until you press Start again. And FORMULAS TRIED genuinely is cross-timeframe: it is carried forward from one shared status file, so a 15m run continues counting from wherever your daily runs left off. Also, a formula re-tried in a later round counts again: the de-duplication cache lives inside a single round only.

BEST FITNESS (the accent tile)

The fitness of the current number-one champion — the same min(TRAIN, VAL) figure explained under Fitness. It is the one tile drawn in the accent colour, because it is the number that says whether the search is getting anywhere. It replaced an older progress chart: one number, written at the end of every round.

HOW TO SET IT

Expect it to jump in the first handful of rounds and then creep. It only moves at round boundaries, so a mid-round silence is not a stall. A long flat stretch means the search has exhausted the easy structure at your current settings — that is the moment to widen the universe, raise the formula complexity limits, or accept the library you have and move to the Portfolio card. It shows "—" until the first round completes, and it deliberately does NOT show a TEST number; the held-out column lives in the leaderboard.

Watch out. If you turned on "Optimize by win rate", this tile still formats the value as a Sharpe-style signed number ("+0.56") instead of a percentage ("56%"). The search itself does switch objective correctly — but the label the node writes into its status file is always 'sharpe', because that one line reads its environment variable under a doubly-prefixed name that never exists. The leaderboard's own fitness column tags each row from the row itself and does show the percentage, so trust the column over the tile in win-rate mode.

The live log (bottom of the status card)

A seven-line, colour-coded feed of what the node is doing right now, fed from the node's own event list (the last 80 events are kept). Round starts and finishes are in normal ink, ★ new-best announcements are green, window fine-tuning is accent-blue, warnings and errors are red, timestamps are faint. It also carries forward-track step messages and one-off notes such as why this install's search is unique.

HOW TO SET IT

This is the single best place to see whether the node is healthy. "► round N: explore" versus "► round N: refine" tells you which mode you are in; "✓ round N · 47s · 4,812 formulas tried · +3 champions kept · best fitness +1.81 · held-out TEST −0.26" tells you the cost and the

yield of a round in one line. If rounds are failing you will see red "X round N failed" lines with the exception name. The box scrolls inside itself and the card never changes size — it is meant to be a constant instrument panel, and it leaves your text selection alone while you are dragging over it.

Watch out. While the library is sealed (that is, before you activate the node), the formula text is stripped out of the log on purpose — the ★ line keeps the numbers but the maths after the em-dash is cut, and fine-tune lines stop at the improved score. Nothing is broken; the vault is doing its job.

SIGNAL API — running services

A card that only appears once you are serving something. Pressing "Serve" on a portfolio (or on a single formula) starts a small local web service that recomputes that strategy's live target positions and hands them out as JSON on localhost. Each service takes the next free port starting at 8799 and gets a row here with its URL, its log file, a health status and a way to shut it down; "× Free all ports" stops all of them.

HOW TO SET IT

This is the hand-off point to your own execution code — poll the URL, get target weights per asset, trade them however you like. The services are deliberately detached processes: they keep serving after you close AlphaNode, and the app re-adopts them on the next launch by probing ports 8799–8808 for a health response. Closing the window asks whether you want them stopped.

Watch out. Because a service outlives the GUI, a stale one from a previous session can still be answering on 8799 with an old strategy while you think you are looking at a fresh one. The card re-adopts whatever it finds on those ports, even services started by an older build (it asks the operating system who owns the port to fill in the missing details). If the numbers look wrong, check the row's label and started-time before blaming the formula.

LEADERBOARD — every alpha, by fitness `min(train,val)`

default: every alpha, sorted by fitness, families-only off; the "Leaderboard size" setting is 20 and does NOT limit this table

The library itself, as a table — not the node's top 20, but every formula ever mined at this timeframe, re-read in a background thread whenever the library file changes (checked every few seconds). Columns: ★ favourite, rank, fitness, TEST OOS, maxDD (worst peak-to-trough loss on TEST), CAGR, sortino, $T \uparrow / T \downarrow / T \sim$ (TEST Sharpe split by whether the market was trending up, down or flat), L/S per asset per year, trades per asset per year, win% (share of the bars it actually traded that ended up) and its up/down-market split, the 6-character id

and the formula. Sortino, L/S and trades-per-year are hidden until you switch them on; the rest of that list is visible by default. The header line changes to tell you which population is currently loaded.

HOW TO SET IT

Click a column to sort. Double-click a row for its equity chart; right-click for copy, CSV export and a column picker. The "families only" switch collapses the table to one representative per formula shape (about 20 of them), which is the fastest way to see how many genuinely distinct ideas you own. The search box filters live by id or formula text. Rows are tinted green only when their held-out TEST Sharpe is at least zero — in a healthy library that is a minority, and that is the honest picture.

Watch out. Two things bite here. Clicking the "fitness" or "TEST OOS" headers does more than re-sort: it re-selects WHICH alphas are loaded from the whole library, and the heading changes to "by TEST OOS — held-out, cherry-picked Δ " to say so; every other column merely reorders what is already on screen. And on a sealed (not yet activated) library, only the numbers stored at mining time survive: fitness, TEST OOS, maxDD and CAGR still show, while sortino, $T\uparrow/T\downarrow/T\sim$, L/S, trades and win% show "—" for every row, because the worker that computes them has no formula to simulate. "Leaderboard size" is not dead either — it caps the node's own champion list, which is also the pool that warm-starts a refine round.

PORTFOLIO — top-N combined

range **top-N:** 2–20 with the spinner arrows (a typed value is not clamped)

default: top 6, by TEST — a widget default only: it is not stored in your settings and "Reset to defaults" does not touch it

Takes N alphas from your library (2–20, default 6), runs them together in a background process and plots the combined equity curve with its metrics above it: Sharpe on TRAIN, VAL and TEST separately, plus TEST CAGR and max drawdown, against a buy-and-hold baseline. The curve spans the whole history, not just TEST. On daily bars this is the full quantpylib simulator — the same engine that Serve and the forward track use. The "by" selector chooses how the members are picked: by TEST Sharpe, by fitness, or "combo" (a greedy search for the best COMBINATION, optimised on TRAIN+VAL only, from a pool of roughly $4 \times N$ alphas, capped at 30).

HOW TO SET IT

This is where the point of mining hundreds of alphas becomes visible: the combined curve is usually far better than any single member, because their mistakes are uncorrelated. But the default "by TEST" is the dishonest pick and the card says so in its own warning line — the same window that chose the members is the window being scored, so that headline TEST number is inflated. Prefer "fitness" or "combo" if you want the displayed TEST number to mean anything; "combo" is slower (it simulates a whole pool, not just N) but hunts

diversification directly. A daily build takes roughly one to two minutes; on 4h/1h/15m it uses the fast approximation instead and finishes in seconds — the line under the chart always names which engine produced the numbers. Once built, the CSV / Serve / PDF / Track buttons wake up.

Watch out. The builder skips every sealed row — it can only simulate formulas it can read. On a library you have not activated, the build stops with "need at least 2 scored alphas in the library", no matter how many hundreds of rows the leaderboard is showing. It also drops near-identical formulas while picking, so "top 6" means the six best DISTINCT shapes, not literally rows 1-6 of the table.

FORWARD TRACK — paper steps, append-only

default: starts from \$10,000 · one step per closed bar of the strategy's own timeframe · checked every 5 minutes

The honest verdict machine. Enrol a portfolio (or a single formula) with "Track" and it is frozen — formulas, universe, target volatility and fee — then paper-traded forward from \$10,000, one step per closed bar: fetch the latest closed candles, recompute target positions with the real engine, mark to market, rebalance, charge fees. History is append-only; nothing is ever recomputed backwards. The table shows steps taken, equity, return, Sharpe, drawdown and the last step's date.

HOW TO SET IT

This is the only number in the app that no amount of peeking can contaminate, because the bars did not exist when the formula was mined. Enrol your best candidates early and give them months — the Sharpe column stays blank until there are enough steps to mean anything, and a dozen steps tell you nothing either way. "Step now" forces a step; otherwise the app checks every 5 minutes while it is open, and the node itself also checks every 5 minutes while it is mining, so the track keeps advancing with the window closed as long as the node runs. "Chart" plots only the live steps; "Signals" gives you the full step-by-step book, rebalances, P&L and fees, exportable as CSV.

Watch out. "Delete" is permanent and takes the paper history with it — re-enrolling the same formula does not resume, it starts a fresh track from a clean \$10,000, so a year of forward evidence is gone in one click. Note also that nothing steps while both the app and the node are shut: if you close AlphaNode for a week, the missed bars are covered by mark-to-market at the next step rather than replayed, and on an intraday track (where bars close every 15 minutes) that turns into a lot of skipped detail — the enrol dialog warns you about exactly this.

Part 02

Pairs, timeframe and market data

These settings decide WHAT the search looks at: which markets, at what bar size, and where the price history comes from. They are the only settings that change the data itself rather than how the data is searched, so get them right before touching anything in SEARCH, GENOME or FITNESS. AlphaNode downloads its own history from Binance USD-M perpetual futures — there is no file to point it at and no Download button. The download starts by itself when you press Start (and once, about a second after the app opens, if the snapshot for the current bar size does not exist at all). Everything downstream — the leaderboard numbers, equity charts, CSV and live signals, the portfolio, the forward track — is computed on exactly the basket and bar size you set here.

Which pairs to trade

default `BTCUSDT,ETHUSDT,SOLUSDT,XRPUSDT,BNBUSDT`

range: No limit is enforced anywhere in the app — you can type as many pairs as you like. (evolution/timeframe.py carries a per-bar-size number — 150 pairs on 1d, 100 on 4h, 60 on 1h, 40 on 15m — but no code reads it, so it is documentation, not a cap.)

The exact basket the whole app runs on — the formula search, the leaderboard's statistics, the equity charts, the CSV and live signals and the forward track all use this list and nothing else. Each pair is a chip; the box under the chips is where you type. Names must be full Binance USDT-perpetual tickers (BTCUSDT, not BTC): what you type is uppercased for you and duplicates are dropped, but nothing appends the USDT for you.

HOW TO SET IT

Editing: Enter, a comma or a space turns what you typed into a chip; pasting a whole list works and splits on commas, spaces, tabs and line breaks; the **x** on a chip removes it; clicking a chip's text pulls it back down into the box so you can fix a typo (it is re-added when you press Enter or click away, but at the END of the list, so the order changes); Backspace in an empty box pulls the last chip down. Empty the field completely and the panel says so in place ('no pairs — Start or Save brings the default five back'), and that is exactly what happens the next time settings are written — you cannot run with no pairs. On size: the cross-sectional operators the search leans on (cs_rank, cs_zscore, cs_demean, cs_scale — they rank or centre one pair against the others on the same bar) are meaningless below about 5-6 pairs, so that is the practical floor. Going wider buys a broader cross-section and more diversification, and costs download time, memory and simulation time in direct proportion — every added pair slows every generation of every round. Adding a recently listed coin is safe: each pair is downloaded from its own listing date and its pre-listing bars stay blank and are masked out of the simulation, so they simply contribute nothing before the coin existed.

Watch out. Two real traps. (1) A one-pair universe looks like it works and does not. The simulator normalises the signal every bar by dividing by the sum of the absolute forecasts, so with a single asset the weight is always exactly ± 1 — only the SIGN of your formula is ever traded, and every alpha collapses to the same long/short flip. (2) Library rows do not record which basket (or which TRAIN/VAL/TEST dates) they were mined on. Change the pairs and one leaderboard row starts mixing two worlds: 'fitness', 'TEST OOS', maxDD and CAGR are the numbers stored when the alpha was mined on the OLD basket, while win%, activity, L/S, sortino, the T columns and the equity chart are re-simulated on the CURRENT basket and current dates. Nothing in the app warns you.

Timeframe (bar size)

default **1d**

range: 1d | 4h | 1h | 15m (fixed dropdown; 5m was removed from the product). An unknown value in a saved settings file falls back to 1d instead of failing.

The bar size for the entire pipeline: search, metrics, signals, portfolio and forward track. Every window inside a formula is counted in BARS of this size, so a 20-bar average is 20 days on 1d and 5 hours on 15m, and Sharpe is annualised from the bar count (crypto trades 24/7, so 365 days, not 252). Each timeframe is a separate world with its own downloaded snapshot (data.pickle for 1d, data_4h / data_1h / data_15m.pickle otherwise), its own alpha library and its own round history — alphas mined on different bar sizes never mix or compete.

HOW TO SET IT

Start on 1d: it has the longest history (from 2019-09-05), the lightest download, the fastest generations, and it is the only snapshot that ships pre-seeded with the installed app. Drop to 4h or 1h when you want more bars per year of calendar time — more observations means a less noisy Sharpe per unit of calendar — but intraday history on Binance perps is shorter, the download is far heavier, and fees and slippage eat a much larger share of each bar's move. 15m is the extreme of that trade. The choice only takes effect when settings are written — press Start, or collapse the settings pane with the header button; until then the leaderboard still lists the previous timeframe's library. Portfolio build and the forward track do work on intraday today, despite the tooltip's line that they are 'daily-only for now': the portfolio quietly switches from the real quantpylib engine to the fast simulator, and the forward track steps once per closed bar of whatever size the strategy was enrolled at. The forward track only advances while the app is open (it checks every 5 minutes), so a track you leave alone overnight catches up in one step rather than one step per missed bar. One thing is NOT isolated per timeframe: 'Clear all history' counts only the active timeframe in its confirmation dialog but deletes every timeframe's library and history — see 'Data and state on disk'.

Watch out. Picking from the timeframe dropdown **OVERWRITES** all four DATE SEGMENTS fields with that bar size's recommended dates, and it does so on any pick from the list — including re-picking the timeframe you are already on. Hand-tuned TRAIN/VAL/TEST dates are thrown away silently. The recommended sets are 1d: 2019-09-05 / 2021-11-01 / 2023-01-01 / 2026-07-05; 4h: 2020-06-01 / 2023-01-01 / 2024-09-01 / 2026-07-01; 1h: 2022-06-01 / 2024-06-01 / 2025-06-01 / 2026-07-01; 15m: 2024-01-01 / 2025-04-01 / 2025-12-01 / 2026-07-01. Always pick the bar size first, then edit the dates.

Market data (the automatic download at Start)

There is no Download button. When you press Start, AlphaNode inspects the snapshot for the active timeframe and downloads first if anything is wrong with it: the file does not exist, it cannot be read, it is missing a pair from your list, or its newest bar is more than 5 days old. It then opens a console window that streams the downloader's live output — the pair list it settled on, a per-pair bar count, rate-limit pauses, and a heartbeat every 20 seconds naming what is downloading right now and what is still queued — and when it exits cleanly Start resumes on its own a moment later. If Binance's live API is unreachable from where you are (a geo-block, for instance), the downloader falls back on its own to Binance's public data archive: the same bars, but roughly 10-30 hours behind live.

HOW TO SET IT

Expect the wait to scale with the bar size: the downloader runs 6 pairs in parallel on 1d (with a 2-minute ceiling per pair), 4 on 4h, 3 on 1h and only 2 on 15m (with a 40-minute ceiling per pair) — deliberately slow, because more parallelism trips Binance's rate limit and stalls everything. Closing the console cancels the download and terminates it; the old snapshot is untouched either way, because the file is written under a temporary name and swapped in only after a complete success. A pair Binance does not serve — a typo, a spot-only coin, a delisted perp — is skipped by the downloader, which still reports success; AlphaNode then drops that pair from your list, saves, and tells you exactly which ones went. Be aware that a pair that merely ran out of time gets the same treatment and the same 'Binance futures does not serve' message, so re-add it and retry before believing it. If a gap somehow survives a download that succeeded, the next Start stops with 'Still ... even after a successful download' and asks you to check the ticker names, rather than looping between download and error forever. Two things worth knowing about freshness: the 5-day staleness rule is counted in DAYS whatever the bar size, so a 15m snapshot can be nearly 500 bars behind and still be accepted as fresh; and the download never asks for a start date, so history always begins at the timeframe's recommended date (1d 2019-09-05, 4h 2020-06-01, 1h 2022-06-01, 15m 2024-01-01) or the pair's listing date, whichever is later — setting TRAIN start earlier than that buys you no extra data, just an empty stretch at the front of your training window. Funding-rate history is fetched alongside the candles, which is what makes the `funding` term inside formulas mean anything.

Watch out. This is a replacement, not a top-up, whatever the panel's own note says. The app asks for exactly the pairs currently in your list and the whole snapshot file is overwritten with just those — so adding one pair re-downloads all of them from scratch (on 15m that is a very long afternoon), and a pair you removed loses its stored history the next time any refresh runs.

Data and state on disk

In the installed app everything writable lives in one folder: `~/local/share/AlphaNode` on Linux, `%APPDATA%\AlphaNode` on Windows, `~/Library/Application Support/AlphaNode` on macOS. It holds the price snapshots (data.pickle plus data_4h / data_1h / data_15m.pickle), your settings (gui_settings.json), an exports/ folder for CSV and PDF output, and a state/ folder with the mined alphas (library.jsonl, library_4h.jsonl ...), the per-round history, status.json, the built portfolio (portfolio.json), the forward track (forward.json), your starred favourites, saved sessions and this install's node id. Running from source instead puts the snapshots at the repository root, settings and state under `alphanode/`, and exports at the repository root.

HOW TO SET IT

On the first run of an installed build, the bundled daily snapshot is copied into that folder once and every later download overwrites it in place — the path never changes, only the contents, which is why fresh data is always picked up. The alphas you have mined are just the library*.jsonl text files: back those up, or use Sessions → Save current..., which packages the formulas, round history, forward track, portfolio and settings into one file, deliberately leaving your licence key out — do that before any destructive action or a move to another machine. Deleting a data_*.pickle is harmless: the next Start downloads it again. Settings, on the other hand, are written only when something triggers a save. Pressing Start does it, so does switching theme, collapsing the settings pane with the header button, and most layout fiddling (resizing or reordering a dashboard card, toggling a leaderboard column, the All/Families switch) — every one of those rewrites the whole settings file, pairs and timeframe included. Closing the window does NOT save, so a pairs or timeframe edit made and immediately followed by quitting is simply lost; tap the settings-pane toggle to commit before you close.

Watch out. 'Clear all history' counts only the active timeframe in its confirmation dialog ('N found alphas, M rounds'), then deletes every library*.jsonl and history*.jsonl it can find in the state folder, plus status.json, portfolio.json and the cached equity images. Clearing while you are on 1d therefore destroys your 4h, 1h and 15m libraries too, and the dialog never says so. Nothing is backed up automatically — save a session first.

Part 03

The search settings — how hard and how long it looks

These are the knobs that decide how hard the app hunts for formulas, for how long, and how it divides its time between looking for brand-new ideas and polishing the ones it already has. The app works in rounds: one round is one complete evolutionary run — a whole population of candidate formulas bred and filtered over a number of generations — and the node repeats rounds until you stop it, adding whatever survived to your library. Everything in this block is read once, at the moment you press Start: changing a value while the node is mining does nothing until you Stop and Start again, and Stop cuts the round in progress short, so anything that round had found but not yet written is lost. None of these settings can touch the honesty of the result — the TEST period is held out of selection no matter what you set here.

Resources (CPU share)

default **50**

range **5–95 (percent; slider only, no typing – and the node re-clamps anything outside 5–95)**

A slider that hands the search a share of your processor. The bold line just above the slider does the arithmetic live — the share is multiplied by your core count and rounded to whole worker processes, never fewer than one, so 50% on an 8-core machine reads "50% → 4 of 8 cores". Each worker is a separate background process that scores candidate formulas at the same time as the others.

HOW TO SET IT

Raise it and rounds finish roughly proportionally faster — this is the single biggest lever on wall-clock speed. What it costs: memory (every worker builds its own copy of the price panel, so RAM grows with the number of workers, not with the percentage), fan noise and heat, and a machine that feels sluggish while it mines. Lower it to about a third if you want to keep working on the same computer. On Linux and macOS the node also asks the operating system for background priority, which softens the sluggishness; on Windows that call is simply skipped. At the very bottom of the range on a small machine the arithmetic lands on one worker, and the engine then runs in a plain single-process mode with no parallel pool at all — same results, one less layer of overhead.

Watch out. The worker pool is created fresh at the start of every round and torn down at the end of it, and each worker rebuilds the price panel from scratch. So a big core share buys you little if Generations is very small — you pay that setup cost on every round instead of spreading it over a long one.

Population

default **200** range **4–4000** (arrows step by 10)

How many candidate formulas exist side by side in one generation. Generation 0 is filled with random formula trees (plus your warm-start champions, if that is on); every later generation is rebuilt from the previous one by selection and breeding.

HOW TO SET IT

Raise it for a wider sweep of formula space on every generation — more building blocks in play, less chance the run gets stuck polishing one mediocre idea. Cost is close to linear in time: a round evaluates at most Population × Generations formulas, usually well under that, because a formula the round has already scored is never scored again (the round-done line in the LIVE LOG reports the true figure as "N formulas tried"). Halving it roughly halves round time and roughly halves the ground covered. If you want more total search, it is usually better to raise Population than Generations, because breadth is what protects you from settling on the first decent idea the run stumbles into.

Watch out. Each new generation is assembled in a fixed order — the elites first (up to Elitism), then the fresh random injections, then bred children until the count reaches Population. Nothing trims it back down, so any Population smaller than Elitism + Random/generation (6 + 10 = 16 with the defaults) is silently overridden: from generation 1 onwards you get about 16 formulas whatever you typed. Two more traps: a number typed into the box is not clamped to 4–4000 (only the up/down arrows respect the limits), and the `population =` line in `evolution/config.ini` has no effect on the app — the node overwrites it with this value on every round.

Generations

default **25** range **1–500**

How many cycles of scoring, selection and breeding happen inside ONE round. Every generation scores the whole population, carries the best formulas over untouched (elitism), breeds new ones by crossing and mutating tournament winners, and sprinkles in fresh random trees. After the last generation there is a short extra pass over the top five champions that fine-tunes their lookback windows: each window in the formula is tried at 0.8× and 1.25× its length, up to three passes, and a change is kept only if it improves the same TRAIN/VAL score the search was already optimising.

HOW TO SET IT

Raise it to let a round dig deeper into whatever it has found — good formulas get combined and refined instead of merely discovered. Cost is linear in time, and the returns flatten. You can watch that happen: the one-line ticker under the status pill shows the current generation and its best score, and the LIVE LOG gets a "★ new best" line each time the record actually improves. If those stars dry up around generation 15, there is little point paying for 40. Lower

it and you get more, shorter rounds, which means more from-scratch restarts and more variety in the library — but you pay the per-round worker-pool and data-preparation overhead more often. Below about 10, evolution barely has time to combine anything and the round is closer to a random search.

Watch out. `evolution/config.ini` says `generations = 30`, and the app never uses that value. The node always overrides population, generations, seed and the number of cores with its own — the settings panel's numbers, or its built-in fallbacks (200 / 25) when it is run headless with no environment set. Only the standalone command-line runner, `evolution/run_evo.py`, reads the config file's `[search]` section (and even there, its `--pop / --gens / --seed` flags win).

Seed (0 = auto)

default **0**

range **0–999999**

The starting number for the random generator that builds and mutates formulas. The same seed with the same data and settings reproduces the same run exactly. Left at 0 the app mints a seed once from a random node ID stored in its state folder (a number between 1 and 900000), so your install walks its own path through formula space and no two installs mine identical libraries.

HOW TO SET IT

Leave it at 0 for normal mining — that is what keeps your library yours. Set an explicit whole number only when you need repeatability: filing a bug, or A/B-testing one other setting with the randomness held still. Be aware that a shared number is a shared search: anyone running the same integer with the same settings and data mines the same formulas. There is no quality difference between seeds; a "good seed" is survivorship bias.

Watch out. Reproducibility only really holds for explore rounds. Each round runs on the seed plus the round number, and refine rounds also start from whatever champions are in memory at that moment — so two machines with the same fixed seed but different libraries diverge from the first refine round onwards. The round counter itself is restored from your history file on restart, so a fixed seed continues past old rounds rather than replaying them.

Pause, sec

default **5**

range **0–3600 (seconds)**

How long the node idles between rounds. It sleeps in half-second slices and checks for a Stop before each one, so Stop still responds during the pause.

HOW TO SET IT

Raise it on a laptop or any thermally limited machine to give the fans a chance to catch up between rounds, or when you want the computer usable in bursts. It is pure lost mining time and has no effect on the quality of what is found — the trade is simply throughput against a cooler, quieter machine. Set 0 to chain rounds back to back.

Watch out. This is not a load limiter. People reach for it to "turn the heat down", but during a round the node still uses every core the CPU-share slider gave it, at full tilt. If the machine is too loud or too busy, lower Resources (CPU share) — Pause only adds dead time between the bursts. Background work (the forward-track stepper, the status web page) keeps running through the pause anyway.

Status port

default 8787

range 1024–65535

The TCP port for the node's own small web page — a live table of the top formulas, the round log and the forward track, at <http://localhost:PORT>. The page reloads itself every four seconds, and the same port also serves the raw numbers at `/status.json`.

HOW TO SET IT

Leave it alone unless something else on the machine already owns 8787, or you want to run two nodes at once (give each its own port). The desktop window does not use this port at all — it reads the node's status file straight from disk — so the page is purely a bonus: a way to check on a mining machine from your phone, another computer, or a headless server.

Watch out. The page listens on all network interfaces, not just localhost, so anyone on the same network who can reach the port can open it. In the sealed retail build the formulas themselves show as "🔒 locked · <id>", but round timings, fitness and the held-out TEST numbers are all in the clear. If the port is already taken the node writes "status server off: ..." to its own console output and simply keeps mining — and the desktop app discards that output, so you get no page, no error dialog and nothing in the LIVE LOG. Also keep the number clear of 8799 and the fifty ports above it: that is where the live-signal services claim their first free port.

Optimize by win rate

default off (unticked)

Switches what the search is trying to maximise. Unticked (the normal mode) it maximises the Sharpe ratio — return per unit of wobble. Ticked, it maximises the per-bar win rate: out of the bars on which the strategy actually held a position, the share that closed in the black (flat bars count as neither a win nor a loss). Either way the number it optimises is the WORSE of the TRAIN and the VAL period, so a formula has to hold up on both.

HOW TO SET IT

The score the search uses is deliberately lower than the raw percentage, and by design: the raw share is first pulled toward 50% by the square root of how often the formula was active (62% of 40 bars is weaker evidence than 55% of 700), and then one binomial standard error is subtracted, computed with a small prior so that a perfect 5-out-of-5 reads as luck rather than certainty. On top of that, any formula that goes under 30 active bars in TRAIN or in VAL is thrown out of the round entirely — and in either mode a formula must be in the market on at least 10% of the bars of both periods or it is discarded as degenerate. Concretely: a formula active every bar that wins 55% of 700 bars scores about 53%; one that wins 60% but only trades 30% of the time scores about 52%. That is also why the fitness never matches the leaderboard's win% column — that column is the raw, undamped share, measured on the held-out TEST period, not the worse of TRAIN and VAL. One more quiet adjustment: because win rate lives near 0.5 while Sharpe spans a range several times wider, the complexity and similarity penalties in the FITNESS section are automatically scaled down tenfold in this mode, so the same numbers there bite about as hard as they do on Sharpe. Use this mode when you want an equity curve that feels steady day to day rather than one carried by a handful of enormous moves, or as a second opinion when every Sharpe champion you mine looks like the same trend-following family. The caveat is not decorative: win rate is blind to size, so 65% small green bars against 35% catastrophic red bars is a losing strategy that this objective happily ranks first. Judge anything it mines by the TEST Sharpe and max drawdown columns before you trust it. Takes effect at the next Start.

Watch out. In this build the switch does not reach the node's own ranking of the library. The node reads its ranking metric as `env('ALPHANODE_FIT_METRIC', ...)` while that helper already adds the `ALPHANODE_` prefix itself, so it looks up a doubled name that nothing ever sets and always comes back 'sharpe'. The search really does optimise win rate and the rows it writes are tagged correctly; only the node's own ordering is stuck on Sharpe. On a library that already contains Sharpe-mined alphas the consequences are: the node's top list (and the :8787 page) keeps those old rows first, the dashboard's fitness tile is drawn from them, and — the expensive one — warm-start refine rounds re-seed from those Sharpe formulas instead of the win-rate champions the run just found. Even on a library with nothing but win-rate rows, that tile prints a 53% score as "+0.53". The library file and the app's own leaderboard table are unaffected (they tag and sort each row by the objective it was mined under, using the checkbox you actually ticked). Until this is fixed, mine win-rate alphas into a fresh library — or into a timeframe whose library is still empty — if you want refine rounds to build on them.

Explore every N-th

default 4

range 1–100

The mix between hunting and polishing. Every round whose number divides evenly by N is an EXPLORE round: a completely fresh random population, looking for formula families you have never seen. Every other round is a REFINE round: the population starts from the champions

currently on the node's top list and evolution mutates around what already works. With the default 4 the pattern is refine, refine, refine, explore, refine, refine, refine, explore... Each round announces which it is in the LIVE LOG — "explore — a fresh random population" or "refine — improving N champions from the library".

HOW TO SET IT

Lower N (2) means more exploring: a broader, more varied library, but your single best fitness creeps up slowly because nothing gets polished for long. Higher N (8) means mostly polishing: the headline number climbs faster, but the library converges onto one or two ideas and eventually plateaus — and a library of eight variations on the same formula is one strategy, not eight. 3 or 4 is the recommended balance. If the top of your leaderboard has been the same formula shape for hours, lower it; if you have hundreds of scattered mediocre alphas and nothing outstanding, raise it.

Watch out. N = 1 quietly turns refinement off completely: every round number is divisible by 1, so every round explores and the warm-start switch below never does anything. The node does print a warning about it, but only to its own console output, which the desktop app throws away — the only visible symptom is that every round in the LIVE LOG says "explore". Separately, a refine round needs readable formulas in memory, and in the sealed retail build the library on disk holds no readable text — so the first round after every start of the app explores regardless of N, and refining only becomes possible once the current session has mined something of its own.

Max. rounds (0=∞)

default 0

range 0–999999

Stop the node automatically after this many rounds. 0 means it never stops on its own — you end it with Stop or by closing the app.

HOW TO SET IT

Use a finite number for a bounded experiment ("50 rounds at population 400, then compare against the same 50 at 200") or an overnight budget you do not want to babysit. Use 0 for a node you simply leave running. Remember it is a count of rounds, not of time: how long a round takes depends on Population × Generations, how many pairs you mine and how many cores you gave it, so the same number can mean twenty minutes or twenty hours.

Watch out. The count is lifetime, not per session. On start the node restores its round number from your history file, so if 120 rounds are already recorded and you set Max rounds to 50, it starts, sees 120 is already past 50, and stops without mining a single formula. Set it above the round count shown on the dashboard — the intuitive reading, "run 50 more rounds", is not what it does. Resetting the counter means clearing history, and "Clear all history" is not a surgical tool: it deletes the round history AND the mined alphas, for every timeframe at once.

Leaderboard size

default 20

range 1–200

How many champions the running node keeps in its own top list. That list is what the :8787 status page shows, what the dashboard's best-fitness tile is drawn from — and, most importantly, the exact pool of formulas that a refine round warm-starts from.

HOW TO SET IT

Treat this as "how wide is my warm start". A small number (5–10) makes refine rounds hammer on a handful of formulas: fast convergence, narrow results. A large number (50+) gives refine a broad, varied starting pool. It also decides how much of an existing library is even eligible: on start the node loads your library, sorts it and keeps only this many rows in memory, so everything below the cut can never seed a refine round again. The cost of large is subtle: the seeds are placed into generation 0 before the random filling, so with a big top list and a small population a refine round can begin almost entirely from old formulas with very little fresh material to breed against. Keep it comfortably below Population — an order of magnitude smaller is a sane ratio.

Watch out. It does not limit your library and it never deletes anything. Every champion ever mined stays appended to the library file, and the app's own leaderboard table reads that file rather than the node's top list, so lowering this number only narrows the warm start and the web page — your history stays intact. (How tall the app's table is has nothing to do with this number: you drag the table's bottom edge to resize it.)

Warm-start from library

default on (ticked)

The master switch for refine rounds. On, a round that is not an explore round begins with your top-list champions dropped into generation 0 unchanged, with the rest of the population filled randomly around them, and evolution improves on them from there. Off, every round is a from-scratch exploration and Explore every N-th stops meaning anything.

HOW TO SET IT

Leave it on for normal use — this is what turns a series of independent searches into something that compounds. Turn it off deliberately when you want breadth rather than a peak: a clean survey of many unrelated formula families, or a fresh look because you suspect your library has converged on a single idea and keeps re-polishing it. With it off, the best fitness you see will climb much more slowly and each round stands on its own.

Watch out. A champion whose formula has more operations than the current Max. nodes (in the GENOME section) is silently skipped when the round seeds itself — no message anywhere. So lowering Max. nodes after mining a batch of large formulas quietly converts your warm start back into a random start. Two smaller points: identical champions are seeded only once, and seeding stops at Population, so a top list larger than the population leaves the tail unused. And in the sealed retail build the formulas stored on disk are not readable, so the warm-start pool only ever contains champions found since the app was last started — restart the app and the first round explores regardless of this tick.

Part 04

The engine knobs — simulation, formula shape, breeding and fitness

These four sections control the search itself: how the simulator trades a formula, how big a formula is allowed to get, how the next batch of candidates is bred, and how candidates are scored. The genetic algorithm behind it works like this — the app holds a population of candidate formulas (200 by default), scores each one on TRAIN and VAL, then builds the next generation by copying the best few through unchanged (elitism), breeding pairs of good ones by swapping pieces of their formulas (crossover), tweaking single ones (mutation), and adding a handful of fresh random formulas; that repeats for the number of generations you set, and the whole thing is one round. TEST is scored and displayed but never used to choose anything, in any of these sections. Every value on this panel is read when you press Start and handed to the mining process, where it overrides the same setting in the app's config.ini — which is why that file's own genome numbers (`max_depth = 8`, `max_size = 40`) never take effect in the desktop app; only settings that have no box on this panel still come from the file. One thing to know about the boxes themselves: they are spinners, and only the up/down arrows respect the limits below. A number you type is passed through unchecked, and a number the app cannot read as a number silently reverts to the shipped default.

Target-vol (ann.)

default **0.25** range **0.01 – 3.00**, arrows step **0.01**

The annual volatility the simulated book is steered to. The engine keeps a running estimate of how much the book itself is swinging and scales every position so that swing lands on this number — 0.25 means "size this formula so a typical year moves about $\pm 25\%$ ". This is the knob that sets leverage: the engine solves for whatever leverage hits the target rather than you typing a leverage figure.

HOW TO SET IT

Because the score being optimised is a Sharpe ratio — return divided by volatility — this knob barely changes which formulas win: double it and both the returns and the swings roughly double. What it changes is the risk you would actually be running, and therefore the reported CAGR and max drawdown. Running the simulator directly on one signal at 0.10 / 0.25 / 0.60 / 1.50 gave realised annual volatility of 0.104 / 0.261 / 0.632 / 1.583 (the targeting works), Sharpe of +0.65 / +0.69 / +0.67 / +0.63 (essentially unchanged), and worst drawdowns of -20% / -44% / -76% / -99.7% . So: set it to the risk you are willing to hold, not as a tuning dial. Costs no extra CPU.

Watch out. The simulator has no margin model and no liquidation, so the high end is arithmetic rather than a tradeable setting. In the same test, the CAGR of that identical, unchanged formula went 6.4% → 15.6% → 25.1% → −21% across those four targets: at 1.50 the Sharpe was still fine and the compounded result was a loss, purely from the size of the swings. If the equity path reaches zero or below, the CAGR column comes back blank (the code returns "no value" rather than a nonsense root for wiped-out capital) while drawdown reads −100%.

Fee (turnover)

default **0.001**

range **0.0000 – 0.0500**, arrows step **0.0005**

What trading costs, as a fraction of the notional that changes hands. 0.001 = 10 basis points = 0.1% of whatever is traded, charged each time the book moves — so a round trip costs about twice this. It is charged on the change in a position, not on the position itself: a formula that sits still pays nothing.

HOW TO SET IT

This is the only brake the search has on churn — there is no separate turnover penalty anywhere in the fitness. Set it at or above your real all-in cost (exchange fee plus expected slippage) and the search will refuse to hand you signals that only work for free. Raise it and the winners get slower and smoother and every reported Sharpe falls; lower it and you get prettier numbers that will not survive contact with an exchange.

Watch out. Setting it to 0 does not just flatter the results, it changes what wins. Measured on the same data: a fast-turning signal scores Sharpe +0.57 with fees off and −0.76 at the default 0.001, while a slow signal only drops from +1.13 to +0.69. An entire family of high-churn formulas becomes "profitable" the moment this is zero, and those are exactly the ones the search will then hand you. In the other direction, note that a hard-coded 10% no-trade band (a position is left alone when the newly wanted size is within 10% of the size already on) already absorbs small rebalances, so your real cost is lower than turnover × fee; that band is not adjustable.

Max. depth

default **6**

range **2 – 12**, arrows step **1**

A formula is a tree: an operator on top, its arguments underneath, raw price/volume features at the bottom. This caps how many operators can be stacked on top of one another — 6 means up to six nested operations before a branch must end in a raw feature such as close, volume or funding.

HOW TO SET IT

This limit binds harder than it looks. Of the 25 operators the search generates, 19 take a single argument (`ts_mean`, `ema`, `cs_rank` and the like) and only 6 take two, so formulas grow as long narrow chains rather than wide bushes — in a real mined library of 270 champions only about one node in ten was a two-argument operator, roughly a third of the champions sat exactly at the depth ceiling, and node counts trailed off well below the node cap. In other words: Max. depth is usually what caps a formula, not Max. nodes. Drop it to 3–4 if you want formulas you can read at a glance; raise it to 8 or so if you want the search to be able to build longer chains of transformations. Each extra level costs evaluation time only through the extra nodes it lets in.

Watch out. Depth is enforced by pruning, and pruning replaces an over-deep branch with a random raw feature — so a child that breaks the limit is not rejected and retried, it is damaged and then evaluated anyway. This bites when you lower the setting: champions loaded from your library to warm-start a refine round are checked for size but never for depth, so the old deep formulas enter the new population intact, and then every child bred from them has its too-deep branches swapped for random leaves.

Max. nodes

default 22

range 3 – 80, arrows step 1

The total count of everything in the formula — every operator and every raw feature. `ts_mean:30(close)` is 2 nodes. Because these formulas are mostly chains of one-argument operators, a formula at the 22-node default is typically around 16–17 operations over 3–5 raw features. This is the overall size limiter, and it is the number the complexity penalty is charged against.

HOW TO SET IT

Small (10–15): simple, readable, harder-to-overfit formulas and a search that converges quickly. Large (40–80): expressive combinations, and many more ways for the search to memorise TRAIN. Evaluation cost rises roughly with node count, since every node is a full computation across every pair and every bar in the date range, on every candidate. If you raise this, raise Complexity penalty too, or you are simply paying formulas to be complicated. Note that in practice evolved formulas often stop well short of this cap and hit Max. depth instead.

Watch out. The limit is not airtight, and it leaks worst exactly where you would care. Crossover and mutation children over the limit are rejected and retried, but the fresh random formulas from "Random/generation" are injected with no size check at all. Measured across 20 seeds: at the default 22 this almost never happens (about one oversize newcomer every twenty generations), but at Max. nodes 8 roughly 4 of the 10 injected formulas per generation are over the limit, the largest seen being 26 nodes. Those oversize newcomers can be selected as parents. Separately, lowering this quietly weakens refine rounds: library champions bigger than the new limit are dropped from the warm start without a word.

Tournament size

default 5 range 2 – 50, arrows step 1

How a parent gets chosen: the algorithm draws this many candidates at random from the current generation and keeps the best of them. 5 means "best of 5 random formulas". This is the single dial for selection pressure — how reliably the good ones get to breed.

HOW TO SET IT

Low (2-3): mating is nearly random, the population stays diverse and explores widely but improves slowly. High (15-50 against a 200 population): the top few per cent dominate at once, fitness climbs fast for two or three generations and then the population becomes copies of one lineage. Watch the per-generation log line "evaluated N (uniq M)": if the unique count stops growing, pressure is too high for your population size. Costs no extra CPU either way.

Watch out. The draws are made with replacement from the same generation, so the odds that the current best is in the tournament climb fast with size: at 50 out of 200 it is in there almost every time and selection has effectively become "always breed the current best". Also, the fitness being compared here is the penalised one — a formula can lose a tournament for being large, or for resembling a Hall of Fame champion, rather than for trading worse.

Elitism

default 6 range 0 – 50, arrows step 1

How many of the best formulas are copied into the next generation completely untouched — no crossover, no mutation. Without it, the best thing you have found can be destroyed by the very mutation that was meant to improve it.

HOW TO SET IT

6 out of a 200 population (3%) is a sane default. At 0, every generation is rebuilt from bred children and progress can go backwards. Above roughly 10% of the population the same handful keeps re-winning tournaments and the search stalls. Elites cost nothing to evaluate (their scores are already in the round's cache), but every elite slot is a slot not spent breeding something new.

Watch out. Elites are picked by the penalised fitness of that moment, which includes the similarity penalty against the Hall of Fame as it stood at the start of the generation. So a formula can be an elite in one generation and drop out of the elite set in the next without changing at all — simply because a new champion that resembles it entered the Hall of Fame in between.

Random/generation

default 10

range 0 – 200, arrows step 1

How many brand-new random formulas are dropped into each generation. Crossover and mutation can only recombine material the population already holds; this is the only channel that reintroduces operators and features the gene pool has lost.

HOW TO SET IT

10 out of 200 (5%) keeps a trickle of novelty without diluting the search. Raise it (20–40) when the log's unique-formula count flattens early, which means the population has converged. Every injection is a full evaluation and nearly all of them are junk, so this is a direct CPU tax: 40 injections in a 200 population spends 20% of your compute on blind guesses.

Watch out. Two real traps here. First, injected formulas skip the Max. nodes filter that crossover and mutation children must pass — harmless at the default, serious at a tight Max. nodes (see that knob). Second, Elitism + Random/generation is not capped by Population: set them to 50 and 200 against a population of 200 and the generation silently becomes 250 individuals (measured), costing 25% more CPU than you asked for. And when those two together already reach Population, no crossover or mutation happens at all — the run degenerates into pure random search with Tournament size and Crossover share doing nothing.

Crossover share

default 0.60

range 0.00 – 1.00, arrows step 0.05

Of the children bred each generation, what fraction are made by crossover — cutting a branch out of one parent and grafting it into a copy of another. The remainder (0.4 at the default) are mutations of a single parent. Mutation is one of three equally likely kinds: replace

a branch with a fresh random one; swap an operator for another of the same kind while keeping its horizon (`ts_mean:30` → `ema:30`); or nudge a lookback window — usually a small jitter around the current one (`30` → `34`, `30` → `24`), and about one time in five a jump to a value from the coarse starting grid (`2`, `3`, `5`, `7`, `10`, `14`, `20`, `30`, `50`, `60`, `100`, `120`, `200`). Windows can end up at any whole number from 2 to 200.

HOW TO SET IT

Crossover recombines whole working ideas, and is how two half-good formulas become one good one; mutation does the fine work, and is the only thing that tunes lookback windows. Push crossover up when the population is already rich and you want fast recombination; push it down when you are refining and want horizons tuned.

Watch out. At 1.00 you switch window tuning off entirely — the window nudge lives only inside mutation, so lookbacks could then change only by grafting in a branch that already had a different one. The only fine tuning left would be the polish pass after evolution ends, which touches just the top 5 champions.

Complexity penalty

default **0.010**

range **0.000 – 1.000**, arrows step **0.005**

Subtracted from a formula's score in proportion to its node count: $\text{score} - \text{penalty} \times \text{nodes}$. At the default that is 0.010 per node, so a 22-node formula gives up 0.22 while the smallest possible 2-node formula gives up 0.02 — a gap of 0.20 in favour of the small one when they score the same. It is the pressure that makes the search prefer the simplest formula that does the job.

HOW TO SET IT

It is charged in the same units as the fitness, so calibrate it against a real Sharpe gap. At the default, the whole spread from the smallest to a full 22-node formula is about 0.2 Sharpe — meaningful, not decisive. At 0.05 a full-size formula gives up 1.1 Sharpe and the search collapses toward stumps. Raise it when champions come out unreadable and their TEST numbers lag their fitness (the classic overfit signature); lower it toward 0.005 when you have raised Max. nodes and want the extra room actually used.

Watch out. This penalty steers selection but not the output. The Hall of Fame — the champion list that gets written to your library — is sorted and deduplicated on the raw, unpenalised score, so a bloated formula with the best raw score still tops your library no matter how high you set this. It changes which parents breed, not which champions you are handed. Separately, with "Optimize by win rate" on, this penalty (and the similarity penalty with it) is multiplied by 0.1 internally, because win rate lives on a ~0.25-wide scale and Sharpe on a ~2.5-wide one — so the same typed number is ten times gentler under that objective.

Correlation threshold

default **0.70** range **0.00 – 1.00**, arrows step **0.05**

How alike two formulas' results must be before the app calls them the same alpha. It compares the correlation of their per-bar TRAIN+VAL returns, ignoring sign; above this line they count as duplicates. It does two jobs at once: it triggers the similarity penalty during selection, and it is the deduplication rule for the Hall of Fame.

HOW TO SET IT

0.70 is a reasonable "same family" line for crypto alphas, which correlate highly by nature. Lower it (0.5–0.6) when your library keeps filling with near-identical variants of one idea; raise it (0.8–0.9) when the search returns too few distinct families and you would rather keep close cousins than nothing. Because the sign is ignored, a formula and its exact inverse count as duplicates of each other.

Watch out. Both ends of the range are broken, in opposite ways. At 0.00 every candidate counts as a duplicate of its nearest champion, so the Hall of Fame rule becomes "replace that one champion if better, otherwise throw the candidate away" — fed 500 completely unrelated candidates, the list ended with exactly one entry (measured), and rounds return almost nothing. At 1.00 the dedup can never fire (correlation cannot exceed 1) and the Hall of Fame degenerates into a plain top-N, i.e. fifteen twins of the same formula; the similarity penalty is switched off at that setting too.

Similarity penalty

default **0.5** range **0.0 – 2.0**, arrows step **0.1**

How hard a candidate is punished for resembling a champion already in the Hall of Fame. The subtraction is $\text{penalty} \times (\text{correlation} - \text{threshold})$, measured against the single most-similar champion. It exists to stop the whole population converging on one idea.

HOW TO SET IT

Raise it when every round returns variations of the same alpha and you want the search pushed into new territory. Lower it to 0 when you are deliberately refining one family and do not want the search shoved away from it. It costs no CPU. Under "Optimize by win rate" it is scaled down tenfold internally, along with the complexity penalty, so it is far gentler than the typed number suggests.

Watch out. At the defaults this knob is close to cosmetic, and it is easy to over-estimate. Because the penalty is proportional to how far past the threshold the correlation sits, the largest subtraction possible is $\text{penalty} \times (1 - \text{threshold}) = 0.5 \times 0.30 = 0.15$ Sharpe — noise next to a champion Sharpe of 1.5–2.5. To make diversity actually cost something you must lower Correlation threshold as well, or run this near its 2.0 maximum. And setting it to 0 does not switch deduplication off: the Hall of Fame still evicts similar champions using Correlation threshold, which is an entirely separate mechanism.

Hall of Fame size

default **15** range **1 – 100**, arrows step **1**

The Hall of Fame is the champion list a round produces — the formulas written into your library when the round finishes. This sets how many it holds. It is not a simple top-N: a new formula that correlates above the threshold with an existing champion replaces that champion if it scores better and is discarded if it does not, so the list is kept diverse rather than merely high-scoring.

HOW TO SET IT

15 per round is already generous. Raise it when rounds keep reporting "none kept — the library held its bar" and you want to see the near-misses; lower it (5–8) when the library is filling with mediocre entries you never open. The cost is real but modest: every formula in the generation is correlated against every champion twice (once for the similarity penalty, once for the Hall of Fame update), so 15 → 100 makes that bookkeeping roughly seven times heavier.

Watch out. Raising it does not get you more of your best work. The polish pass that runs after evolution ends — the one that fine-tunes champion lookbacks by trying $\times 0.8$ and $\times 1.25$ on each horizon — is hard-coded to the top 5 champions regardless of this setting. And champions already in your library are skipped when the round writes its results (matched by the formula text), so a larger Hall of Fame mostly adds weaker, newer entries rather than better ones.

Robust blocks (0 = legacy)

default **0** range **0 – 12**, arrows step **1**

An alternative way of scoring a formula. At 0 (the default), fitness is $\min(\text{TRAIN Sharpe}, \text{VAL Sharpe})$ — a formula is only as good as its worse half. Set it to 2 or more and the app instead merges TRAIN and VAL into one span, cuts it into that many equal consecutive time slices, shrinks each slice's Sharpe by its own margin of error (short or merely lucky slices lose the

most) and scores the formula on the near-worst slice — with the shipped setting and 3 blocks, on a value halfway between the worst and the middle one. The intent is that a formula must work across regimes rather than shine in one.

HOW TO SET IT

The honest answer first: the app's own config.ini records a retrospective A/B on the 861-alpha July daily library and it found no improvement. The rank correlation between fitness and held-out TEST was +0.40 for the legacy rule, +0.40 for the best 3-block variant, and +0.26 for 5 blocks — finer slices only added per-block noise. The reason is written in the same comment: min(TRAIN, VAL) already is a two-block worst-regime rule, split at the bull/bear boundary. Treat this as an opt-in experiment; if you try it, use 3, not 5 or more. It costs no extra CPU.

Watch out. Turning it on quietly changes what the word "fitness" means everywhere, and nothing in the interface says so. (1) The numbers stop being comparable: each slice takes a margin-of-error haircut of roughly a full Sharpe point when a 3.3-year selection span is cut into 3, so block fitness is routinely deep in the negative — the same three formulas scored $-0.28 / -0.32 / -0.89$ under the legacy rule and $-0.86 / -1.92 / -1.26$ at 3 blocks (measured) — yet the leaderboard's fitness tooltip and the "best fitness" tile still read "min(TRAIN, VAL) Sharpe". (2) Worse, the leaderboard sorts on that raw number without knowing which rule produced it, so in a library that already holds legacy rows the block-scored champions land underneath all of them, get trimmed off the leaderboard (20 rows by default) and never seed a warm-start refine round — new champions reach the library file and then stop influencing the search. (3) A candidate whose selection span is shorter than blocks $\times$ 30 bars is thrown out as invalid, so a short custom TRAIN+VAL plus a high block count can make every candidate fail and the round return nothing. (4) The two companion settings — how far into the worst blocks to reach (block_quantile, 0.25) and how much margin of error to subtract (se_penalty, 1.0) — live only in config.ini and have no box on this panel; the concentration guard in that same file (conc_penalty 0.3, min_eff_n 3 — docking fitness from books that average under 3 effective positions) is likewise off-panel, is independent of this switch and stays on either way. One more: if you upgrade from a version that predates this setting, your saved settings file has no entry for it and the box opens showing 5 rather than 0 — check it reads 0 before your first Start, or press "Reset to defaults".

Part 05

Date segments, and the honesty discipline

AlphaNode slices your price history into three consecutive stretches — TRAIN, then VAL, then TEST — and four dates in Settings define all three. The search is allowed to stare at TRAIN and VAL as long as it likes; TEST is walled off and only ever reported, never used to pick, rank, seed or fine-tune anything. That single rule is the difference between a number you found and a number you fitted: an alpha's score is the WORSE of its TRAIN and VAL Sharpe ratios (Sharpe = average return divided by how much that return wobbles, scaled up to a yearly figure — crypto trades every day, so the engine annualises over 365 days, not 252; higher means a smoother climb), so a formula has to work on both halves, and TEST then tells you whether that survived contact with years it had never seen. This section covers what each of the four dates does, what a sensible cut looks like at each bar size, and how to move the segments forward as time passes without quietly cheating yourself.

TRAIN start

default **2019-09-05**

range: a plain text box, YYYY-MM-DD — nothing checks the format, and nothing checks that the four dates are in order

The very first bar the app loads, for any purpose at all. Nothing before this date exists as far as AlphaNode is concerned — the price table, the rolling volatility estimate and the one continuous simulated equity curve all begin here. It is also the start of the training stretch: the data the search is free to fit as hard as it likes.

HOW TO SET IT

Earlier means more history and more market moods for a formula to prove itself across, at the price of speed: every candidate is simulated over the entire TRAIN-start-to-TEST-end span on every single evaluation (TEST included — it is simulated, just never scored), so doubling the span roughly doubles the cost of a round. There is also a warm-up tax at this edge. A formula may look back up to 200 bars; operators that compare today against N bars ago produce nothing at all until N bars have passed, while rolling ones (average, standard deviation, high/low, ranking) start halfway through their window — a 200-bar average begins at bar 100. On top of that the first five bars are never tradable for anybody, and the rolling volatility estimate needs 30 bars (180 at 4h, 720 at 1h, 2,880 at 15m) before it means anything — until then every coin is sized off the same floor value, so the opening stretch does trade, just crudely. Leave a couple of hundred bars of runway before the part of TRAIN you actually care about. Because the simulation is one unbroken run whose volatility state carries forward from the very first bar, changing TRAIN start also nudges the TEST numbers very slightly — that is not leakage, just the same strategy started from a different point.

Watch out. The download that Start fires off never asks for your TRAIN start. It calls the fetcher without a start date, so you get the timeframe's own recommended history date (2019-09-05 for 1d, 2020-06-01 for 4h, 2022-06-01 for 1h, 2024-01-01 for 15m). Set TRAIN start earlier than that and nothing warns you — the pre-Start data check only verifies that your pairs are present and that the newest bar is no more than 5 days old, never that the snapshot reaches back far enough. The missing stretch gets filled with a frozen, ineligible price, so every formula sits flat through it; flat bars deliberately stay in the scoring window, so every Sharpe is dragged down by roughly the square root of the live share of the window, and if less than 10% of TRAIN (or of VAL) is left trading, the activity filter rejects every candidate and the round finds nothing at all. A second, quieter version of the same trap: clear one of these boxes entirely and the node does not complain — an empty field silently falls back to whatever that date is set to in `evolution/config.ini`.

VAL start

default **2021-11-01**

range: a plain text box, YYYY-MM-DD — it should sit between TRAIN start and TEST start, but nothing enforces that

The wall between TRAIN and VAL. VAL is the second stretch the search is allowed to see, and its job is to be a second opinion: an alpha's score is the LOWER of its TRAIN Sharpe and its VAL Sharpe, so a formula that made all its money in one kind of market is judged by the kind where it did not.

HOW TO SET IT

This is the most powerful anti-overfitting lever in the whole app, and it is a date, not a number. Put it where the market's character genuinely changes — the shipped 1d cut lands it at the top of the 2021 bull run, so TRAIN is the run-up and VAL is the bear market that followed, and 'the worse of the two' becomes a blunt does-this-work-in-both-worlds test. Move it later and TRAIN gets richer while VAL thins into noise; move it earlier and VAL quietly becomes the dominant judge. Keep VAL long enough that its Sharpe means something: the engine's own error-bar formula for a Sharpe, $\sqrt{(1 + S^2/2) / \text{years}}$, puts about ± 1.2 around a one-year VAL measured at Sharpe 1.0. That error only shrinks with the square root of length — two years brings it to ± 0.87 , four years to ± 0.61 — so lengthening VAL helps, but slowly. A VAL shorter than a year is a coin toss wearing a decimal point.

Watch out. One experimental setting silently retires this date from the score. Set 'Robust blocks (0 = legacy)' in the FITNESS section to 2 or more (it ships at 0, and the box accepts 0-12), or the equivalent blocks value in config.ini, and the engine stops using the worse-of-TRAIN-and-VAL rule entirely: it re-cuts the whole span from TRAIN start to TEST start into that many equal slices and scores a near-worst slice instead. The VAL boundary plays no part in the number any more. It still decides validity — a formula that trades on under 10% of TRAIN or of VAL, or that has too little to measure on either, is still thrown out — and the VAL row still shows on the champion card, so nothing looks different while the number you are optimising has changed meaning. Two more traps in the same box: 1 is not 'one block', it silently falls back to the legacy score with no message; and if the TRAIN-to-TEST span holds fewer than 30 bars per block, every candidate is rejected and the round finds nothing. It ships off for a reason — config.ini records a re-run on an 861-alpha daily library that found no improvement over the plain worse-of-two rule.

TEST start

default **2023-01-01**

range: a plain text box, YYYY-MM-DD — it should sit between VAL start and TEST end, but nothing enforces that

The wall. Everything from this date to TEST end is held out — it never enters the score, never ranks the champion shortlist, never seeds a warm-start round, never guides the fine-tuning of a champion's lookback windows. Its numbers are computed and carried along purely so you can read them afterwards. It is also the far end of the selection span: the multi-block score, when switched on, and the position-concentration check both run from TRAIN start to TEST start.

HOW TO SET IT

Everything after this date is your one honest read, so treat its length as a budget you are spending. Push it later and the search gets more data while your verdict gets noisier; pull it earlier and you buy a long, trustworthy TEST at the cost of a thinner search. The shipped 1d split is deliberately lopsided in TEST's favour — roughly 790 TRAIN bars, 430 VAL bars and 1,280 TEST bars, so TEST is the biggest of the three at about half the total history. For a daily search, keep TEST to at least 1.5-2 years of calendar. Make TEST too short and it degrades quietly rather than loudly: under 5 bars where the strategy actually traded, the TEST numbers come back empty and the TEST OOS cell just shows '—'; the market-direction columns (T ↑, T ↓, T ~, win ↑, win ↓) each need 30 bars in their bucket and blank out first; and sorting the leaderboard by TEST OOS drops every row with no TEST number out of the list entirely, so a too-short TEST makes alphas appear to vanish.

Watch out. Re-cutting this date leaves the library in mixed vintage, and nothing on screen says so. The fitness and TEST OOS cells are frozen into the library file the moment an alpha is mined, under whatever dates were in Settings then — and maxDD and CAGR reuse that stored value whenever the row has one. The stored row records which objective it was mined under, but not the segment dates. Meanwhile sortino, $T \uparrow / T \downarrow / T \sim$, win%, win $\uparrow / \downarrow$ and the L/S and tr/yr activity rates are recomputed live by the background metrics worker against whatever dates are in Settings right now (it is handed TRAIN start, TEST start and TEST end — VAL start is not even sent). Move TEST start and an old row will show you a TEST Sharpe from the old window sitting next to direction statistics from the new one.

TEST end

default **2026-07-05**

range **a plain text box, YYYY-MM-DD – nothing clamps it to the data you actually hold**

The last bar the app loads, for everything — search, leaderboard numbers, portfolio build, CSV export. It is also the end of TEST.

HOW TO SET IT

Normally this should be today, or the last complete bar in your snapshot. There is no reason to leave downloaded history unused, and the fetcher defaults its own end date to today, so the data is almost always ahead of this field.

Watch out. Two real traps sit here. First, setting it past the end of your data costs you Sharpe silently: the price table carries the last close forward over the empty stretch, so the price never moves, those bars drop out of eligibility and every strategy returns exactly zero across them — and because flat bars deliberately stay in the scoring window, TEST Sharpe gets multiplied by roughly the square root of the share of real bars, with CAGR deflated harder still (the same total return, spread over more claimed years). Every alpha in the library takes the same invisible haircut. Second, the shipped default is a fixed date, not 'today'. Start refreshes your snapshot whenever its newest bar is more than 5 days old, so the data keeps moving while TEST end does not — leave it alone for a few months and the most recent stretch of real, downloaded market history is simply never looked at.

Timeframe (bar size)

default **1d**

range **1d | 4h | 1h | 15m**

Picks the bar size for the whole pipeline — search, leaderboard numbers, live signals. Each timeframe keeps its own data snapshot and its own alpha library, so alphas mined on different bar sizes never mix. Choosing one from the dropdown also fills all four date boxes

with that timeframe's recommended cut.

HOW TO SET IT

The four recommended cuts: 1d — 2019-09-05 / 2021-11-01 / 2023-01-01 / 2026-07-05, about 790 / 430 / 1,280 bars over 6.8 years. 4h — 2020-06-01 / 2023-01-01 / 2024-09-01 / 2026-07-01, about 5,660 / 3,650 / 4,010 bars. 1h — 2022-06-01 / 2024-06-01 / 2025-06-01 / 2026-07-01, about 17,500 / 8,760 / 9,480 bars. 15m — 2024-01-01 / 2025-04-01 / 2025-12-01 / 2026-07-01, about 43,800 / 23,400 / 20,400 bars. The pattern is deliberate: finer bars trade calendar span for bar count, because intraday history on Binance perps is shorter, far denser and much heavier to download. The trap to internalise is that the extra bars do not buy you a more believable verdict — the engine's own error bar on a Sharpe, $\sqrt{(1 + S^2/2) / \text{years}}$, depends on calendar YEARS, not on how many bars you sliced those years into. At Sharpe 1.0 the shipped TEST windows carry roughly ± 0.65 (1d, 3.5 years), ± 0.91 (4h, 22 months), ± 1.18 (1h, 13 months) and ± 1.60 (15m, 7 months). A 15m TEST holds 20,000 bars and still cannot tell a Sharpe of 1 from a Sharpe of 0. Read intraday TEST as a sanity check, not as evidence, and lean much harder on the forward track there.

Watch out. Picking a timeframe from the dropdown overwrites all four date boxes on the spot, with no confirmation and no undo. Switch to 4h to glance at something and switch back to 1d, and your hand-tuned 1d cut is gone — replaced by the shipped 1d dates. 'Reset to defaults' does the same thing (it keeps only your theme and whether the settings pane is open, and deletes the saved settings file). If your cut matters, save a session first (☐ Sessions), because a session snapshot includes the settings file and is the only way back.

fitness

The leaderboard column showing the number the search actually maximises: the lower of the alpha's TRAIN Sharpe and its VAL Sharpe. Taking the lower of the two is what makes it a robustness score rather than a best-case one — an alpha is worth only as much as its worse segment.

HOW TO SET IT

This is the column to leave the leaderboard sorted by, and it is the honest ordering because it was computed without TEST ever being consulted (it is also where the table opens every launch). The displayed value is the raw lower-of-two; during the run the engine additionally subtracts a small complexity charge (the complexity penalty times the number of operations in the formula), and, for a candidate that closely resembles a champion already banked this round, a similarity charge on top — but those are ranking aids inside the generation and are not baked into the stored number. With 'Optimize by win rate' switched on the cell shows a percentage instead of a Sharpe: that mode maximises the lower of TRAIN and VAL of a win rate that has been shrunk for how much evidence stands behind it, which lives on a completely different scale.

Watch out. A library accumulates rows mined under different settings, and a row records which objective it was mined under but not the dates. Two rows both reading '+1.42' may have been measured on entirely different years — after any re-cut, fitness is no longer comparable across rows, and nothing in the table tells you which is which. The objective tag does have one visible effect: clicking 'fitness' never produces a single ladder when the library mixes Sharpe-mined and win-rate-mined rows. Every row mined under the objective currently selected in Settings sorts above every row mined under the other one, whatever the numbers say — the two are different units and one raw ladder would drown the whole of one mode.

TEST OOS

The alpha's Sharpe on the held-out stretch — the one number in the app that no part of the search ever optimised, seeded from, or ranked by. Rows whose TEST came out at or above zero get the green tint; that tint is the only colour signal on the table, and it marks survivors, not winners.

HOW TO SET IT

Read this as a verdict, not as a ranking key. Expect it to come out below fitness — that is normal and not failure; a wide gap between a glowing fitness and a limp TEST OOS is the classic overfitting tell, and a row where the two are close is worth more than a row where TEST alone is high. Pair it with maxDD and the market-direction columns before you believe anything: a TEST Sharpe that lives entirely in $T \uparrow$ is a long-the-market proxy, not an alpha.

Watch out. Clicking the 'TEST OOS' header does far more than reorder the rows in front of you — it re-selects the whole population from the entire library by that column and discards every alpha with no TEST number at all. The card title changes to 'LEADERBOARD — every alpha, by TEST OOS — held-out, cherry-picked $\triangle$ ', and it means exactly what it says. Picking the top row of that list is choosing on TEST, which turns your one held-out number into an in-sample one, and there is no way to un-see it. The re-selection is not saved — the next launch opens on fitness again — but that undoes the sort, not the peek. Sort by fitness, choose there, then read TEST as the answer.

by (portfolio member selector: TEST / fitness / combo)

default TEST (it is not stored in settings at all – the app opens on TEST every launch, with top-N at 6)

range TEST | fitness | combo

How the top-N alphas that go into the combined portfolio are chosen from the library. This is where the TRAIN/VAL/TEST discipline is easiest to break by accident, because combining alphas is a selection step just like any other.

HOW TO SET IT

'fitness' picks members by the worse of TRAIN and VAL — TEST never enters the choice, so the combined TEST equity curve is genuinely out-of-sample and you can quote it. 'combo' is better still: instead of the N best individuals it searches for the best COMBINATION of N. It takes a pool of the top alphas by fitness (four times N, but never fewer than 12 or more than 30), simulates the whole pool once, then runs a greedy-then-swap search that maximises the mix's Sharpe on TRAIN+VAL only. TEST stays out, and because the goal is the mix's Sharpe rather than any member's, the search naturally hunts members that do not move together. It is slower, because the whole pool has to be simulated rather than just the N you keep. 'TEST' picks members by their held-out Sharpe and hands you an optimistic curve.

Watch out. The selector opens on TEST — the one option of the three that breaks the discipline — and the panel prints its own warning underneath the chart before you have even built anything. The part of a TEST-picked portfolio that is still real is the diversification gain: combined being far better than any single member is a structural fact about blending signals that do not move together, and that survives the cherry-pick. The level of the combined curve does not.

Clear all history

Deletes every mined alpha and every round record, plus the current status, the built portfolio and the saved equity images, after a confirmation dialog that counts what will go. Search settings and starred favourites survive, and so does the forward track.

HOW TO SET IT

This is the clean way to act on a re-cut. Once you move TEST start or TEST end, the honest options are to wipe and re-mine on the new cut, or to treat the surviving library as historical and never compare its numbers with new rows. In practice: mine under one cut, read TEST once, enroll whatever you liked into the forward track — an enrolled entry freezes its own dates and settings at enrolment, so it keeps accumulating genuinely unseen bars no matter what you do to the date fields afterwards — then push the segments forward and clear. Save a named session before you press it: a session snapshots the whole workspace including the settings file, so both the old library and the old date cut can be brought back. The button also refuses to run while the node is still going.

Watch out. It is not per-timeframe, but the dialog reads as if it were. The counts it shows you are for the ACTIVE timeframe's library and round history only, while the delete itself sweeps every library and history file in the folder — so clearing while you happen to be on 15m destroys your 1d, 4h and 1h libraries too, after showing you a dialog that mentioned only the 15m row counts.

Part 06

The leaderboard column by column — and what you can do with a formula

The leaderboard is the list of every alpha (trading formula) your node has mined into the current timeframe's library. Four of its columns — fitness, TEST OOS, maxDD and CAGR — are stored numbers copied straight from the library row, written when the formula was found. The rest are recomputed on demand by a background helper process that re-simulates the formula on the TEST slice only, using your CURRENT pairs list, target vol, fee and TRAIN-start / TEST-start / TEST-end dates; that is why those cells fill in a moment after the rows appear, and why new ones fill in as you scroll down to them. A single dot (·) in a cell means "still computing, wait a second"; an em dash (—) means "there is no honest number here" — either the formula could not be simulated at all, or that particular statistic had too little evidence to report. One thing to know about the first few seconds after launch: reading and ranking the whole library happens in the background, and until it finishes the table shows only the short top list the node itself publishes (as many rows as the "Leaderboard size" setting, 20 by default). Once the scan completes, the table switches to the full library.

★ (star column)

default **always shown; cannot be hidden**

The first, narrow column. A star means this formula is in your Favorites list; an empty cell means it is not. Clicking anywhere inside this one cell toggles the star immediately — no dialog, no confirmation. It is also in the right-click menu as '★ Star / unstar'.

HOW TO SET IT

Use it as your shortlist while you scroll a library of hundreds of rows: star anything worth a second look, then work through the ★ Favorites window later instead of re-finding rows. Starring costs nothing and changes nothing about the search.

Watch out. The star set is keyed by the formula's 6-char ID alone, with no timeframe in the key, so if the exact same formula text exists in both your 1d and your 4h library, starring it in one shows it starred in the other. Sealed (locked) rows cannot be starred at all — there is no formula text to keep, so you get a message in the card heading for about a second instead of a star.

#

default **always shown; cannot be hidden**

The row's position in whatever order the table is currently sorted — 1 is the top row of the current sort, not a permanent rank. Re-sort the table and every number here is rewritten.

HOW TO SET IT

Treat it as a place-marker for talking about rows ("the third one down"), nothing more. It is never a quality score, and it is not sortable — this header has no click action at all.

fitness

default **always shown; cannot be hidden**

The score the search ranked formulas by. By default it is the Sharpe ratio (average return divided by the wobble of those returns, annualized — higher means a smoother climb) measured on TRAIN and on VAL, with the WORSE of the two taken: a formula only scores well if it works on both halves of the in-sample data.

HOW TO SET IT

This is the only column that is honest for picking: TEST never touched it. Sort by it (the default sort, best first) when you want to know what the search believed in. As a rough working rule on daily bars, anything above roughly +1.5 is worth opening; well above +2.5 is usually too good and worth suspecting rather than celebrating.

Watch out. Two things the 'min(TRAIN, VAL)' tooltip does not tell you. First, a concentration guard is ON by default (config.ini: conc_penalty 0.3, min_eff_n 3) and it subtracts from this number whenever the formula's book averages fewer than 3 meaningful positions per bar — so a one-coin formula shows a fitness below its actual min(TRAIN,VAL) Sharpe, at the shipped defaults, with no blocks setting touched. Second, the number shown is the score BEFORE the search's own size penalty (parsimony, 0.010 per node) and its similarity penalty; the engine subtracts those when it decides which formula wins a tournament, but it stores and displays the unpenalized score. Separately: if you set 'Robust blocks' to 2 or more, this becomes a per-block quantile of the whole TRAIN→TEST span instead, which is not comparable with the default number. And with 'Optimize by win rate' on, the cell shows a percentage instead of a Sharpe, and clicking this header always sorts rows mined under your ACTIVE objective to the top as a block before flipping order inside each block — because a 0.57 win rate and a +1.8 Sharpe are not on the same ladder.

TEST OOS

default **always shown; cannot be hidden**

The formula's Sharpe ratio on the held-out TEST slice — data the search never saw and never optimized against. This is the honest out-of-sample verdict on a single formula. A dash means there was too little to measure: fewer than 5 TEST bars on which the strategy actually made or lost anything, or a return series with essentially no variation.

HOW TO SET IT

Read it, do not select on it. A green row tint marks every row whose TEST Sharpe came out at zero or better — that minority is the interesting set. Use TEST to sanity-check a formula you found by fitness, and expect TEST to sit well below fitness; a formula whose TEST is close to its fitness is the rare good sign.

Watch out. Clicking this header does far more than re-sort. It switches which formulas the table contains: the whole library is re-read and re-ranked by held-out TEST, every row with no TEST number is dropped from the table entirely, and the card heading changes to 'by TEST OOS — held-out, cherry-picked $\triangle$ '. That is peeking, by construction — you are now choosing formulas with the data you were holding out. Click 'fitness' to get the honest population back.

maxDD

default shown by default

Worst peak-to-trough fall of the equity curve on TEST, as a negative percentage — if this reads -34%, the strategy at some point lost 34% from its previous high before recovering. The pain metric.

HOW TO SET IT

This is what decides whether you could actually hold the thing. Two formulas with the same Sharpe and drawdowns of -15% vs -45% are completely different products. Drawdown moves with your target-vol setting: run the same alpha at half the target vol and the drawdown comes down roughly in step.

Watch out. Unlike sortino, win% and the T columns, this cell prefers the value STORED in the library row when the alpha was mined, and only falls back to a fresh computation for older rows that lack one. So if you have since changed target vol, fee, the pairs list or the date splits, maxDD and CAGR still describe the old settings while every other recomputed cell in the same row describes the new ones. Sharpe barely moves when you change target vol (returns and fees scale together); drawdown and CAGR move a lot — which is exactly where the two silently disagree.

CAGR

default shown by default

Compound annual growth rate on TEST — the yearly percentage the TEST equity curve grew at, if you smoothed it into a constant rate. A dash here means the equity ended at or below zero, so no growth rate exists.

HOW TO SET IT

Pair it with maxDD, never read it alone: CAGR divided by |maxDD| is a quick sense of return-per-unit-of-pain. Like maxDD it moves with your target-vol setting, so it says as much about your risk dial as about the alpha's edge — Sharpe is the scale-free version of the same question.

Watch out. Same stale-value trap as maxDD: taken from the library row first, so it can be a number produced under settings you have since changed.

sortino

default **HIDDEN** by default – turn it on with right-click on any column header → sortino

Like Sharpe, but only the losing bars count toward the 'wobble' in the denominator — upside volatility is free. On TEST. A strategy that jumps around a lot but only jumps upward scores much better here than on Sharpe.

HOW TO SET IT

Look at it when a formula's Sharpe seems unfairly low: a big sortino-to-Sharpe gap means the volatility Sharpe punished was mostly good volatility. If sortino is barely above Sharpe, the returns are symmetric and Sharpe was telling the truth.

Watch out. A dash here can mean the opposite of failure: when a formula has no losing bars at all on TEST the downside deviation is zero and the code returns nothing rather than infinity. In practice that almost always means an alpha that traded a handful of bars, not a flawless one — cross-check tr/yr·a before getting excited. (A dash also appears when the whole row failed to simulate, in which case all nine recomputed cells dash together.)

T ↑

default **shown by default**

The formula's TEST Sharpe measured only on bars where the market as a whole was trending UP. 'Trending up' is decided by the market itself, not the formula: an equal-weight basket of your universe has its drift measured over the trailing ~30 calendar days of bars, and if that drift is statistically confident (t-statistic of 1.28 or more, about 90% one-sided confidence) the bar is labelled with the drift's sign. The header carries the sample size, e.g. 'T ↑ ·196' = 196 TEST bars fell in this bucket.

HOW TO SET IT

This is the regime question: does the alpha only make money when everything is going up (in which case you have bought beta, not alpha), or does it work in both directions? A formula with T↑ and T↓ both meaningfully positive is genuinely direction-neutral and much more

valuable than one whose whole edge is in the $T \uparrow$ bucket. Analysis only — filtering or picking by these columns is another layer of TEST peeking.

Watch out. The regime label is deliberately LAGGED one bar: a bar is judged by the regime that was already known at the previous bar's close. Without that lag a plain market-clone showed roughly +1.3 of pure $T \uparrow$ -minus- $T \downarrow$ Sharpe out of thin air, because each return was being sliced by a window containing itself. On dashes, note there are two different causes: fewer than 30 TEST bars in the bucket (the split is the market's calendar, identical for every row, so that kind of dash hits every row at once), or the formula sat flat through the entire bucket so its returns there have no variation at all — that kind is per-row, and it is why one row can dash here while its neighbours show numbers.

$T \downarrow$

default **shown by default**

The same measurement on bars where the market basket was confidently trending DOWN. Same ~30-calendar-day drift test, same one-bar lag, same sample size stamped in the header.

HOW TO SET IT

The most informative of the three for crypto. Most formulas quietly make their money long; a positive $T \downarrow$ is the evidence that the alpha survives a bear leg. If you plan to run this through a drawdown, this column is the one to read first.

$T \sim$

default **shown by default**

TEST Sharpe on FLAT bars — the ones where the basket's drift was not statistically confident in either direction (t-statistic below 1.28). Usually the largest of the three buckets.

HOW TO SET IT

This is where mean-reversion and cross-sectional relative-value formulas earn their keep, and where trend-following formulas usually bleed. A formula strong in $T \sim$ and weak in $T \uparrow/T \downarrow$ is a chop specialist — useful, but it will underperform exactly when the market is most exciting.

L/S /yr·a

default **HIDDEN by default** — turn it on with right-click on any column header → L/S /yr·a

Two numbers separated by a slash: long entries and short entries, per asset, per year, on TEST — e.g. '3.2/1.8' means roughly three new long positions and two new shorts per coin per year. An 'entry' is the moment the target weight crosses into long or short from flat or from the opposite side (the threshold is a weight of 0.0005 in absolute terms). 'Per asset' divides by the coins that were actually tradable during TEST, so the number is comparable across universes.

HOW TO SET IT

Read it as the alpha's directional personality. Something like 4.0/0.1 is a long-only formula wearing a long/short costume — it will die in a bear market whatever its TEST Sharpe says. Something near 2.5/2.5 is genuinely two-sided. Because it is normalized per asset per year, you can compare a 5-coin universe against a 30-coin one directly.

Watch out. Flipping straight from long to short counts as ONE entry (the short one), and holding a position for eight months counts as zero further entries. So this number measures decision frequency, not how much notional you push around — a slow formula that swings hard between big long and big short books can show a small L/S while turning over a lot.

tr/yr·a

default **HIDDEN by default** – turn it on with right-click on any column header → tr/yr·a

Trades per asset per year on TEST — simply the two L/S numbers added together. Shown with one decimal below 10 and rounded to whole numbers above it.

HOW TO SET IT

The activity dial. Low single digits means a slow position-holder where fees barely matter; a few hundred means an alpha whose profitability lives or dies on your fee setting, so re-check TEST Sharpe with a realistic fee before believing it.

Watch out. It counts INTENDED entries, not fills. The count is read off the formula's target book, while the simulator that produces every return number in the row leaves a position alone whenever the required change is under 10% of the new target size. So this column is an honest measure of how often the formula changes its mind, and an overstatement of how often you would actually touch the exchange.

win%

default **shown by default**

The share of bars on TEST where the strategy made money. Crucially the denominator counts only ACTIVE bars — bars where the strategy actually had profit or loss; bars where it sat flat are excluded entirely.

HOW TO SET IT

Best read against $\text{tr/yr}\cdot\text{a}$. A high win% on an alpha that trades constantly is a real edge; a high win% on an alpha that trades rarely is usually a small sample dressed up as skill. Win rate and profitability are different questions — a 45% win rate with big winners beats a 65% win rate with big losers.

Watch out. The header tooltip says 'share of profitable days', but on the 4h / 1h / 15m timeframes these are bars, not days, and there are many more of them. And because flat bars are excluded from the denominator, a formula active on 8% of bars can post an impressive win% off a tiny sample — this is exactly the inflation the win-rate SEARCH objective corrects for, by damping the edge by how much of the calendar the formula actually traded and then subtracting a margin of error. Do not read a win-rate-mined row's fitness cell as a corrected version of this cell, though: the fitness percentage is measured on TRAIN and VAL, this column is measured on TEST. They are different periods, not two views of the same number.

win ↑

default shown by default

Win rate measured only on bars where the market was confidently trending up — the same market-direction regime, with the same one-bar lag, as the $T \uparrow$ column.

HOW TO SET IT

Use it with $T \uparrow$ to separate two different failure modes: a decent win↑ with a poor $T \uparrow$ means the formula wins often but its losses in up-markets are large; a poor win↑ with a decent $T \uparrow$ means it wins rarely but wins big. Both are real strategies; they need very different position sizing and very different patience from you.

Watch out. This one has a stricter evidence bar than the T columns: it needs at least 30 TEST bars in the regime AND at least 5 of those bars where the formula actually made or lost something. So a dash here on a row whose $T \uparrow$ shows a number just means the formula was mostly flat during up-trends.

win ↓

default shown by default

Win rate on confidently trending-DOWN market bars, same regime definition and same two evidence floors as win ↑.

HOW TO SET IT

The stress-test companion to $T \downarrow$. If win↓ collapses while win↑ stays high, the formula is a long-bias machine no matter what its long/short entry counts suggest.

ID

default shown by default

A short, stable 6-character fingerprint of the formula (the first six characters of the md5 hash of its text). The same formula always gets the same ID, and nothing else does.

HOW TO SET IT

This is the handle you use everywhere else: the Forward Track panel lists single-alpha entries under this exact ID, Favorites shows it, the search box above the table matches on it, and both CSV exports carry it. Copy it when you want to talk about a formula without pasting the whole expression.

Watch out. The two exports do not print it the same way. 'Export table (CSV)' writes the same 6 characters you see; 'Export full library (CSV)' writes 12 characters, of which the first 6 are the ID on screen — so a plain text match between the two files will fail unless you trim. Sealed (locked) rows show the first 6 characters of the vault's public ID, which is computed from the formula text the same way, so a row's ID does not change when a subscription unlocks it.

formula

default always shown; cannot be hidden

The alpha itself, in full — the actual expression the engine evaluates, written in the primitive language the search builds from. The column is sized to the widest formula in the table rather than truncated, so long ones make the horizontal scrollbar appear.

HOW TO SET IT

Formula length is itself information: while ranking candidates the search subtracts a penalty proportional to the number of nodes in the tree (0.010 per node by default), so a short formula that still made the library beat that handicap and is usually the more robust one. Click any cell to make its text selectable, or use Copy formula / Ctrl+C to take the whole thing.

Watch out. Because the simulator normalizes the signal every bar (it divides by the sum of absolute forecasts), the formula's absolute SCALE is meaningless — a formula and the same formula multiplied by 100 are literally the same strategy and would produce identical rows. Do not read big constants in an expression as 'aggressive'. A locked row shows 'locked — a subscription reveals the formula' here; its nine recomputed cells (sortino, the T columns, L/S, tr/yr, the win rates) all show dashes because the helper process has no text to simulate, but fitness, TEST OOS, maxDD and CAGR still show real numbers, because those were stored in the row before it was sealed.

Double-click a row / 'Show equity'

Double-clicking any row (or right-click → Show equity) opens the equity window for that formula: growth of \$1 on a log scale, shaded into TRAIN | VAL | TEST, with an equal-weight buy-and-hold basket of your universe drawn alongside for comparison. All curves are net of fees. Stretches where market volatility was above its own one-year median are tinted, and a lower panel tracks the unrealized profit of whatever positions were open at the time. The header repeats Sharpe / CAGR / drawdown per segment, plus the T↑/T↓/T~ and win-by-direction lines if they have already been computed for that row.

HOW TO SET IT

This is the real test of a leaderboard row: you are looking for a curve that keeps its shape across the TRAIN→VAL→TEST boundaries rather than one that made all its money in one lucky stretch. The chart is a live matplotlib canvas — mouse-wheel to zoom, drag to pan, and the toolbar resets it.

Watch out. A double-click that lands inside the ★ cell never opens the chart — each of its two clicks toggles the star, so the star ends up back where it started and nothing appears to happen. If a double-click does nothing, you hit the first column.

★ Favorites

The button in the leaderboard heading opens your starred list: date added, ID, which timeframe it came from, its fitness and TEST OOS, and the formula. Double-click a row there to open its equity chart, exactly like the leaderboard; Delete or the 'Remove ☆' button unstars.

HOW TO SET IT

The point of Favorites is durability. The starred formulas live in their own file outside the library, so they survive 'Clear all history' and survive loading a different session — they are the one thing in the app that is safe from a workspace wipe. Star anything you would be upset to lose.

Watch out. A favorite is a snapshot: the whole row is copied into the favorites file at the moment you star it. The fitness and TEST OOS shown in that window are the values as of that day, and they are never refreshed — if you later change target vol, fees or the date splits, the leaderboard's numbers move and the Favorites window's do not.

Chart (Forward Track panel)

Select an enrolled strategy in the Forward Track table and press Chart (or double-click the row) to see its forward equity curve: one point per live paper step since enrollment, with the starting capital drawn as a dashed line.

HOW TO SET IT

This curve is the only one in the whole app that was never fitted to anything — every point was computed after the fact on data that did not exist when the formula was found. It is short and noisy for weeks, and that is fine; the value is that it cannot lie to you. Judge a candidate here before you judge it anywhere else.

Watch out. The history is append-only and nothing is ever recomputed backwards: if the app was closed for a week, the gap simply stays as a gap in the line. On intraday timeframes, where steps only happen while the app is running, that makes the track genuinely sparse.

Passport (in the equity window)

Builds an explanation of the formula rather than a measurement of it: the expression drawn as a tree with human-readable labels, plain-English reading steps, the formula's signal plotted against one asset's actual price, an ablation showing which inputs it really depends on, its holding period and long/short balance, and which classic strategy archetype its returns most resemble. No network, no AI — it is a fixed translation of the operators.

HOW TO SET IT

Use it when a formula looks good and you want to know WHY before you trust it — an alpha you can describe in a sentence ('it fades three-day volume spikes') is one you can reason about when it stops working. Takes a few seconds to build.

Watch out. An explanation is not evidence, and the window says so on purpose. A convincing story about a formula does not make its TEST number any more real — the ordering is always TEST first, forward track second, story third.

PDF report (in the equity window)

Renders a four-page analytics dashboard as a PDF file you choose the location of: headline numbers with equity and drawdown; exposure, long/short balance and turnover over time; the structure of the weights plus a monthly-return calendar; and a TRAIN/VAL/TEST breakdown with attribution and written conclusions. Every page is stamped with the date, the time and the formula's 6-char ID.

HOW TO SET IT

This is the artefact to keep or hand to someone else — the app's own state can be cleared, but a PDF is dated evidence of what a formula looked like on a given day with given settings. The Portfolio card has its own PDF button that does the same for a combined top-N book.

Serve signal (API) (in the equity window)

Starts a small local web service that keeps this formula's live target positions available as JSON on your own machine. It fetches fresh closed candles from Binance for your universe, recomputes the target weights, and serves the latest bar at /signal (plus a /health endpoint that reports its age and what it is currently doing). Each service claims the next free port starting from 8799 and appears in the SIGNAL API card, with its URL, its log, and a button to free the port.

HOW TO SET IT

This is the bridge to an execution bot: point your own code at `http://127.0.0.1:<port>/signal` and read the positions list. The service is deliberately given the same target vol and fee as the leaderboard, so what it serves matches what you were looking at. It refreshes every 15 minutes on 1d and 4h bars, every 5 minutes on 1h and 15m.

Watch out. The response carries 'leverage' as a separate field from each position's 'weight' on daily bars — there the weights are the relative book and you must multiply by leverage to get real position sizes. On the intraday timeframes the weight already includes leverage and the served leverage is a flat 1.0, so the arithmetic 'weight × leverage × equity' is what stays identical between the two, not the weights themselves. Also: serving the same alpha twice does nothing new — the service's name is derived from the formula ID, so the second attempt just re-shows the running one.

Download signals (CSV) (in the equity window)

Writes the formula's full position history to a CSV: one row per asset per bar on which it held a meaningful position (anything under 0.0005 of the book is dropped as dust), labelled with its segment (TRAIN / VAL / TEST), the ticker, LONG or SHORT, the signed weight both as a number and as a percentage, and that asset's open/high/low/close/volume on that bar. A dialog then shows what the strategy would be holding on the very last bar.

HOW TO SET IT

This is how you take the strategy out of AlphaNode — backtest it in your own tooling, feed it to a broker script, or just eyeball whether the positions make sense to a human. The last-bar dialog is the quick 'what do I buy right now' answer.

Watch out. The weights in this file are the normalized target book — the absolute values across a bar sum to 1, i.e. 100% of the book — and they are taken BEFORE the engine's volatility targeting. They are relative allocations, not position sizes. If you size positions directly from this column you are running the strategy at whatever leverage you happened to pick, not at your target vol.

Forward track + (in the equity window)

Enrolls this alpha into the forward track: the strategy is frozen exactly as it stands right now — formula, universe, target vol, fee, timeframe — and from then on it is paper-traded once per closed bar on freshly downloaded live Binance data, starting from a clean \$10,000. It offers to run the first step immediately.

HOW TO SET IT

Do this the moment a formula survives TEST and looks explainable. Everything else in the app is measured on data that already existed; this is the only clock that runs forward. Give it weeks before you read anything into the number.

Watch out. A single-alpha entry takes the leaderboard's own 6-char ID as its identity, so enrolling the same formula twice is refused with 'Already enrolled' rather than starting a second track — and the same refusal catches a re-enrollment with an identical formula set, universe and timeframe even under a different name. Delete on that panel removes the strategy AND its whole paper history permanently; re-enrolling starts over from \$10,000. On 4h/1h/15m the app warns you at enrollment: steps only happen while the app is open, and fees weigh proportionally more on short bars.

Copy formula / Copy formula + metrics

Right-click a row for both. 'Copy formula' (also Ctrl+C with a row selected) puts the bare expression on the clipboard. 'Copy formula + metrics' adds a second line with the fitness value, a note if it was mined by win rate, and the TRAIN, VAL and TEST Sharpe ratios.

HOW TO SET IT

Use the plain copy when you're pasting into code or a formula field, the +metrics version when you're recording a candidate in notes or a message — that one line carries enough context to be readable months later. The card heading flashes a confirmation.

Right-click a column header → the Columns menu

default maxDD, CAGR, ID, T↑, T↓, T~, win%, win↑, win↓ on; sortino, L/S, tr/yr off

Right-clicking on the header strip (rather than on a row) opens a checklist of the twelve optional columns: maxDD, CAGR, ID, sortino, T↑, T↓, T~, L/S, tr/yr, win%, win↑, win↓. Tick or untick to show or hide. Your choice is saved and reused next launch. The four core columns — ★, #, fitness, TEST OOS — and the formula are always present and cannot be switched off.

HOW TO SET IT

The default set deliberately leaves sortino, L/S and tr/yr off to keep the table readable. Turn L/S and tr/yr on the moment you start caring whether an alpha is genuinely two-sided or how much its fee assumption matters; turn the T and win-by-direction columns off if you only care about headline performance.

Watch out. Column order is fixed and follows the menu, not the order you tick things in — the visible set is always laid out as ★, #, fitness, TEST OOS, then whichever of maxDD, CAGR, ID, sortino, T↑, T↓, T~, L/S, tr/yr, win%, win↑, win↓ are on, in that sequence, then formula. So with the shipped defaults ID sits between CAGR and T↑ rather than at the far right, and turning sortino on inserts it between ID and T↑ rather than at the end. Hiding a column only hides it: sorting and both CSV exports carry every column regardless.

Sorting (click any column header)

default sorted by fitness, descending

Clicking a header sorts by that column, best first; clicking the same header again flips the direction, and a small arrow marks the active column. Every column except ★, # and ID is sortable — including formula, which sorts alphabetically.

HOW TO SET IT

Sorting by fitness (the default) and by TEST OOS change which formulas the table contains, not just their order — see the TEST OOS entry. Every other column simply reorders the rows you already have.

Watch out. Three things. First, sorting by any recomputed column (sortino, the T columns, L/S, tr/yr, win%, win↑, win↓, maxDD, CAGR) forces the app to compute those statistics for EVERY row currently in the table, not just the visible ones — otherwise the order would be wrong. On a large library showing every alpha that is a long, heavy job with a stalled-looking table. Second, rows with no value for the sorted column are treated as minus-infinity, so descending sort pushes the dashes to the bottom as you'd expect, but ascending sort puts them all at the TOP. Third, unlike the families switch, your sort choice is not remembered: every launch starts back at fitness, descending.

Export table (CSV)...

Right-click menu. Writes exactly the rows currently in the table, in the current order, to a CSV with twenty-two columns — every leaderboard metric plus the TRAIN and VAL Sharpe ratios and which objective the row was mined under — regardless of what is hidden on screen.

HOW TO SET IT

This is the export for a specific working set: filter with the search box or sort the way you want, then export just those rows with all their analysis columns.

Watch out. The recomputed columns (sortino, $T\uparrow/T\downarrow/T\sim$, long/short, tr/yr, the win rates) are read from an in-memory cache that only ever holds rows which have actually been on screen — and it fills roughly a screenful at a time. Export a 600-row table without scrolling and everything past the first screenful comes out with those cells blank. Scroll through the rows you care about first — or sort by one of those columns, which forces the whole set to be computed. Fitness, TEST OOS, maxDD and CAGR are unaffected; they come from the library row and are always written.

Export full library (CSV)... / the CSV button

The 'CSV' button in the leaderboard heading and the matching right-click item both do this: dump every alpha the node has ever mined into the current timeframe's library — no family dedup, no TEST filter, no on-screen subset — sorted by fitness, best first. Nineteen columns: ID, formula, genome size, fitness, the round it was found in, its timestamp, its origin, and the stored Sharpe / drawdown / CAGR / active-bar count for each of TRAIN, VAL and TEST.

HOW TO SET IT

This is the archive export: use it when you want to do your own analysis outside the app, or to keep a record before clearing history. It reads the library file directly, so it works even while the node is mining (a half-written last line is skipped).

Watch out. It carries NONE of the recomputed columns — no sortino, no regime splits, no trade counts, no win rates — because those are not stored in the library, only computed on demand. If you need those, use 'Export table (CSV)...' instead. Also, this file sorts strictly by the raw fitness number, so in a library that mixes both objectives every win-rate row (a value under 1) sinks below every Sharpe row regardless of quality.

Sessions → Save current...

The  Sessions button in the app header opens a list of workspace snapshots. 'Save current...' asks for a name and packs the whole workspace into one file: every timeframe's formulas, the forward track and its history, the built portfolio, and your settings.

HOW TO SET IT

Save one before anything irreversible — clearing history, loading another session, or a big change to the search settings. The list shows how many alphas and how much forward equity each snapshot holds, and double-clicking a row shows the full details including the portfolio and every setting it was saved with.

Watch out. Your licence key is deliberately never written into a session file, so a snapshot is safe to move or share; the machine it lands on will need its own activation.

Sessions → Load selected

Replaces the entire current workspace with the selected snapshot's, then rebuilds the window around the restored settings.

HOW TO SET IT

Use it to switch between lines of work — a 1d research workspace and a 15m one, say — or to roll back to a known-good state.

Watch out. Two hard edges. It refuses to run while the node is searching, a forward-track step is in flight, or a portfolio build is going, because a child process finishing after the swap would silently overwrite the restored files — wait for it to finish. And nothing is backed up automatically: the app never snapshots on its own, so the current workspace is simply gone unless you saved it first. The forward track resumes from the next closed bar, leaving the interruption visible as a gap.

Clear all history

The red button in the settings panel. Deletes the mined alpha libraries, the round-by-round history, the current status file, the built portfolio, and the cached equity images, after showing you a count of the alphas and rounds about to go.

HOW TO SET IT

The confirmation dialog is honest about what survives: your search settings on the left, and your starred favorites. Everything else is gone irreversibly, so save a session first if there is any chance you want it back. It refuses to run while the node is mining.

Watch out. It clears EVERY timeframe, not just the one you are looking at — the delete sweeps every library and history file in the folder, so a wipe launched from the 15m view also destroys your 1d, 4h and 1h libraries. Worse, the counts in the confirmation dialog are read from the CURRENT timeframe only, so a dialog saying '40 found alphas' can be about to delete several thousand. This is the single most destructive button in the app and neither its label nor its dialog says so.

► Build portfolio

default top 6, by TEST (not persisted – restored on every launch)

range top: 2–20; by: TEST | fitness | combo

Takes N alphas from the library and runs them together as one combined book through the simulation engine, then shows the combined equity curve with Sharpe for TRAIN, VAL and TEST, plus CAGR and max drawdown on TEST, against an equal-weight buy-and-hold basket.

The 'by' selector decides how the N members are chosen: TEST picks the best held-out performers, fitness picks by $\min(\text{train}, \text{val})$, and combo searches for the best COMBINATION of N rather than the N best.

HOW TO SET IT

The real prize here is diversification: a combined book routinely beats any single member, because the members' mistakes are not the same mistakes. Prefer 'fitness' or 'combo' — both keep TEST out of the selection, so the combined TEST number you are shown is genuinely out-of-sample. 'combo' additionally hunts for members that are uncorrelated with each other, because that is what maximizes the mix's Sharpe on TRAIN+VAL; it is slower, since a pool of roughly 4N candidates (never fewer than 12, never more than 30) is simulated rather than just N. Once a build finishes, four buttons unlock: CSV (combined signals), Serve (one API for the whole book), PDF, and Track (enroll the whole portfolio into the forward track).

Watch out. The 'by' selector opens on TEST, which is the one dishonest option — the same window that picked the members is then used to report their combined performance, so the headline number is inflated by construction. The card says so in its own note. If you build by TEST, treat the combined Sharpe as unusable and validate on the forward track instead; the diversification gain is still the real part. Note also that neither 'top' nor 'by' is saved anywhere: both reset to 6 and TEST on every launch, so a habit of building by fitness has to be re-established each session.

'families only' switch (families vs all)

default **off** (every alpha shown)

The toggle in the leaderboard heading. Off (the shipped default) the table shows every alpha in the library. On, it collapses to the best representative per 'family', where two formulas count as the same family if their text is 80% or more similar — the compact view that hides a hundred near-identical variants of one idea.

HOW TO SET IT

Turn it on when the search has converged and the top of your table is fifty cosmetic rewrites of the same expression; you want the distinct ideas, not the duplicates. Turn it off when you want to see the whole population, or when you are searching by ID. Both sets are computed in the background and cached, so the switch is instant and sorting works identically in either mode. Unlike the sort order, this choice is remembered across restarts.

Watch out. Families mode is capped twice over: it stops after collecting 20 distinct families, and it only ever examines the top 500 rows of the library while doing so (comparing every formula against every other is quadratic and would otherwise freeze the window). So a genuinely novel formula sitting at rank 600 is invisible in this mode no matter how different it is — 'families only' is a view of the head of the list, not a summary of the library.